

第一讲 数据挖掘的基本认知

信管2302于景琪
202321054034

阅读《参考教材1》的第1-2章、4-5章。

- 第一章 认识大数据挖掘

- 核心知识点

1. 数据定义与特征：大数据是规模巨大、蕴含丰富信息的海量数据，核心特征用4V概括——Volume（体量巨大，TB/PB级）、Variety（类型繁多，含日志、音视频等）、Value（价值密度低）、Velocity（处理速度快）。
2. 数据与数据挖掘的关系：大数据的应用分支中，数据挖掘是核心技术，是数据分析的高级阶段，涵盖数据管理、统计分析，可结合并行/分布式计算，能解决复杂的大数据应用问题，类比煤矿挖掘，大数据是“巨型矿山”，数据挖掘是“采矿技术”。
3. 数据挖掘的概念与原理：数据挖掘是从大量、不完全、有噪声的数据中提取隐含的、潜在有用信息的过程，基于数据库、统计学、机器学习等多学科融合，通过算法训练模型，刻画数据输入与输出的关系。
4. 数据挖掘的六大核心内容：关联（发现变量间规律性，如“尿布与啤酒”）、回归（定量分析变量间关系）、分类（按特征科学归类）、聚类（无先验知识下按相似性分组）、预测（基于历史数据预判未来）、诊断（识别离群点，如欺诈探测）。
5. 应用领域与挖掘要点：应用覆盖零售、银行、证券、能源等多行业；核心要点包括树立大数据思维、重视数据收集与集成、合理进行数据降维、采用分布式与并行处理。

阅读《参考教材1》的第1-2章、4-5章。

- 第二章 数据挖掘的过程及工具

- 核心知识点

1. 据挖掘过程（DPEMED模型）：

- ① 定义目标：明确业务问题、数据范围及预期效果（如3个月内收益提升5%）。
- ② 数据准备：含数据选择（内部+外部数据，涵盖交易、关系、人口统计类）、数据质量分析（检测缺失值、错误值等）、数据预处理（清洗、集成、归约、变换），是耗时最多的环节。
- ③ 数据探索：通过描述统计、数据可视化等初步分析数据特征，与数据准备形成双向反馈。
- ④ 模型建立：选择算法（关联、回归等六大类），划分训练集与测试集，常用简单验证、交叉验证等方法验证模型。
- ⑤ 模型评估：结合精确度、LIFT、ROC等指标，兼顾商业价值与可解释性，而非仅追求数学完美。
- ⑥ 模型部署：将验证通过的模型部署到业务系统，可通过MATLAB等工具部署到多语言开发环境，且需持续监控优化。

2. 用数据挖掘工具对比：

- ① MATLAB：擅长矩阵计算、算法开发，有丰富工具箱，适合学习、研发与灵活产品开发。
- ② SAS：大数据处理能力强，支持二次开发，符合工业行业标准。
- ③ SPSS：界面友好、操作简单，适合初学者，能解决大部分统计学问题。
- ④ WEKA：Java开源工具，含主流算法，适合有开发经验的工程师二次开发。
- ⑤ R：开源、函数丰富，适合算法学习与小项目开发。

阅读《参考教材1》的第1-2章、4-5章。

- 第四章 数据的准备

- 核心知识点

1. 据收集：

- ① 数据属性与存在形式：属性分为离散型（有限/可数值）和连续型（数值型）；数据形式包括结构化（数据库）、非结构化（音视频）、半结构化（邮件、HTML）。
 - a. 数据源：所有与业务目标相关的数据均可作为数据源，实践中常从数据仓库或数据集市获取，金融行业典型数据源包括交易数据库、财务报表数据库等。
 - b. 数据抽样方法：简单随机抽样、系统抽样、整群抽样、分层抽样，需保证抽样的代表性与随机性。
 - c. 数据获取实例：通过MATLAB接口从雅虎获取交易数据、从大智慧/Wind获取财务数据。

2. 据质量分析：

- ① 目的：保证数据的正确性（无缺失、错误）与有效性（含足够信息量）。
 - a. 方法：值分析（统计唯一值、空值占比等）、统计分析（均值、方差、偏度等）、频次与直方图分析（直观呈现数据分布）。

阅读《参考教材1》的第1-2章、4-5章。

● 第四章 数据的准备

① 数据预处理：

1. 数据清洗：处理缺失值（删除法、均值/回归插补等）、过滤噪声（回归法、均值平滑法等）。
2. 数据集成：将分散数据源逻辑/物理集中，含内接、外接等表拼接方式。
3. 数据归约：通过属性选择（删除无关属性）、样本选择（抽样）缩减数据量。
4. 数据变换：含标准化（0-1标准化、Z-score标准化）、离散化（连续数据分段）、语义转换（字符型转整型）。

阅读《参考教材1》的第1-2章、4-5章。

- 第五章 数据的探索

- 核心知识点

1. 据探索的目的与特点：

- ① 目的是为建模做准备，明确变量与样本选择；特点是让数据说话、方法灵活、结果直观，无需过多先验假定。

2. 生变量：

- ① 定义：由既有变量组合衍生而来（如密度=质量/体积），能更直观反映事物特征。

- ② 原则与方法：需客观反映特征且关联业务目标，方法包括变量组合、分类汇总、变量分解、提取时序特征等。

- ③ 类型：含指标型（如股票MACD、RSI）与评价型（用于评价事物好坏，如股票优劣评级）。

3. 据的统计分析：

- ① 基本描述性统计：通过均值、中位数、标准差、偏度等统计量，定量反映数据位置、散度、分布形状。

- ② 分布描述性统计：通过分布函数、分位数、柱状图等，呈现数据概率分布特征。

4. 据可视化：

- ① 基本可视化：用plot等绘制变量分布趋势，了解数据基本特征。

- ② 分布形状可视化：通过hist绘制柱状图，判断数据集中程度。

- ① 关联情况可视化：用plotmatrix、corrcoef等呈现变量间关联趋势与强度。

- ② 分组可视化：用boxplot绘制箱体图，观察数据分布与异常值。

阅读《参考教材1》的第1-2章、4-5章。

- 第五章 数据的探索

- ① 本选择与数据降维：

1. 样本选择：常用随机、顺序、监督取样法，需保证样本数量充足且平衡性（如分类样本各类别数量一致）。
2. 数据降维：主成分分析（PCA）通过线性组合将多变量转化为少数互不相关的主成分，保留85%以上信息；相关系数法根据相关性强度筛选变量。

2、阅读《白话机器学习算法》第1章，整理读书笔记。

核心知识点

一. 据科学研究四步流程：

1. 准备数据：处理数据格式（表格为常用形式，行是数据点、列是变量），区分二值、分类、整型、连续四类变量；通过变量选择筛选核心变量，借助特征工程（如变量重新编码、降维）优化数据，采用近似（众数/中位数填充）、计算（监督学习估算）、移除（谨慎使用）三种方式处理缺失数据。
2. 选择算法：按任务类型分为无监督学习（挖掘隐藏模式，如k均值聚类）、监督学习（基于已有模式预测，含回归和分类问题）、强化学习（通过反馈持续优化）三类。
3. 参数调优：调整算法参数避免过拟合（过度敏感随机波动）或欠拟合（忽视基本模式），可通过正则化引入惩罚参数平衡模型复杂度与准确度。
4. 评价模型：分类任务用预测准确率、混淆矩阵，回归任务用均方根误差；通过验证评估泛化能力，含划分训练集与测试集、交叉验证（数据集分组反复训练测试）两种方式。

2、阅读《白话机器学习算法》第1章，整理读书笔记。

核心知识点

二．关键概念辨析：

1. 过拟合vs欠拟合：过拟合对当前数据预测准但泛化差，欠拟合则忽视重要趋势，均会降低预测效果。
2. 训练集vs测试集：训练集用于生成调整模型，测试集模拟新数据评估准确度，需随机分配保证无偏差。
3. 交叉验证：解决小数据集难以拆分训练/测试集的问题，最大化利用数据，结果更可靠。

3、整理本章上页PPT“本章概念汇总”，形成笔记。

一．大数据（Big Data）

1. 又称海量数据，指的是以不同形式存在于数据库、网络等媒介上蕴含丰富信息的规模巨大的数据。

① 核心特征（4V）：Volume（体量巨大，达TB/PB级）、Variety（类型繁多，含日志、音视频、地理位置等）、Value（价值密度低，需挖掘提取有效信息）、Velocity（处理速度快，与传统数据挖掘有本质区别）。

二．数据挖掘（Data Mining）

1. 是从大量的、不完全的、有噪声的、模糊的、随机的实际应用数据中，②提取③隐含在其中的、人们事先不知道的，但又是④潜在有用的信息和知识的过程。

① 本质：交叉学科，融合数据库、统计学、应用数学、机器学习、可视化等多领域技术，通过算法训练模型刻画数据关系，最终服务于决策。

② 核心内容：关联、回归、分类、聚类、预测、诊断六大类任务。

③ 与大数据的关系：数据挖掘是大数据应用的核心技术，是数据分析的高级阶段，涵盖数据管理、统计分析，可结合同行/分布式计算处理复杂大数据问题。

3、整理本章上页PPT“本章概念汇总”，形成笔记。

三．数据降维（主成分分析PCA、t-SNE）

- 在保留数据核心信息的前提下，减少数据维度（变量个数）的技术，用于解决高维数据带来的计算复杂、冗余等问题。
 - 核心目的：简化模型、提升计算效率、避免过拟合、便于数据可视化。
- 常用方法：
 - 主成分分析（PCA）：通过线性组合将多变量转化为少数互不相关的主成分，保留85%以上信息，是无监督降维的经典方法。
 - t-SNE：基于数据点间的相似度映射到低维空间，擅长保留数据的局部结构，适合高维数据可视化（如高维特征的样本聚类展示）。

四．有监督（supervised）

- 基于已有模式进行预测的机器学习方式，训练数据包含输入特征和对应的标签（已知结果），模型通过学习特征与标签的映射关系实现预测。
 - 核心任务：回归（预测连续值，如房价、销售额）、分类（预测二元值或分类值，如疾病诊断、垃圾邮件识别）。
 - 典型算法：回归分析、k最近邻、支持向量机、决策树、随机森林、神经网络。

3、整理本章上页PPT “本章概念汇总”，形成笔记。

- 无监督 (unsupervised)
 - 无需已知标签，依靠算法从数据中自主发现隐藏模式的机器学习方式，不依赖人工标注的训练样本。
 - 核心任务：聚类（按数据相似性分组）、关联规则挖掘（发现数据间关联关系）、降维（简化数据维度同时保留关键信息）。
 - 典型算法：k均值聚类、主成分分析（PCA）、关联规则（Apriori算法）、Louvain方法。

结构化数据 (Structured Data)、半结构化数据 (semi- Structured Data)、非结构化数据 (Non-Structured Data)

数据类型	定义	典型事例
结构化数据	具有固定格式和组织形式，符合数据库存储规范的数据	企业ERP数据、财务报表、银行交易记录、关系型数据库表
半结构化数据	无严格固定格式，但存在一定结构特征，可通过标签等识别信息	邮件、HTML页面、报表、资源库、XML文件
非结构化数据	无预定义格式和结构，难以直接用传统数据库存储和处理的数据	视频、音频、图片、图像、文档、文本、社交媒体动态

3、整理本章上页PPT “本章概念汇总”，形成笔记。

六. 数据预处理

- 1. 数据挖掘中耗时最多的环节，是数据准备的核心步骤，旨在将原始数据转化为高质量、适合建模的数据。
 - ① 核心目的：保证数据的准确性、完整性和一致性，为后续挖掘提供可靠基础。
 - ② 主要环节：数据清洗、数据集成、数据归约、数据变换。
- 数据清洗
 - 1. 处理原始数据中的错误、缺失值、异常值等问题的过程，是数据预处理的基础。
 - ① 处理对象：缺失值（空值、无意义编码）、数据错误（输入排字错误）、度量标准错误（不正确度量方案导致的数据）、编码不一致（非标准单位、矛盾值）、异常值（偏离正常分布的数据）。
 - ② 常用方法：缺失值处理（众数/中位数填充、监督学习估算、谨慎移除）、异常值过滤（统计法、距离法）、编码统一化。
 - 2. 数据标准化（Normalization）、数据归一化

概念	定义	核心目的	常用方法
数据标准化	将数据转换为均值为0、标准差为1的标准正态分布形式	消除不同变量度量单位的影响，使变量具有可比性	Z-score标准化 $((x-\mu)/\sigma)$
数据归一化	将数据压缩到指定区间（常用[0,1]或[-1,1]）	避免数据量纲差异导致的模型权重偏差，加速算法收敛	最小-最大归一化 $((x-\min)/(\max-\min))$

3、整理本章上页PPT“本章概念汇总”，形成笔记。



◎ 衍生变量 (derived variable)

- 由已有原始变量通过组合、转换、计算等方式生成的新变量，能更直观、精准地反映事物特征。
- 生成原则：客观贴合业务目标，与挖掘任务强相关，避免冗余。
- 生成方法：变量组合（如“维生素C-脂肪”）、分类汇总、变量分解、提取时序特征（如股票MACD、RSI指标）、评价型衍生（如产品优劣评级）。





YOUR
LOGO

数据挖掘与 机器学习

作业二

信管2302 于景琪
202321054034





YOUR
LOGO

01

整理数据回归方法的基本方法、步骤 (J p125)



数据回归方法的基本方法、步骤



按因变量个数与函数类型划分

一元线性回归、一元非线性回归、多元线性回归、多元非线性回归



特殊回归方式

逐步回归（回归过程中可动态调整变量数）、Logistic回归（以指数结构函数为回归模型）

01

数据收集

搜集包含因变量（待预测/解释变量）和自变量（影响因素）的相关数据，确保数据覆盖研究场景的核心特征

02

模型选定与参数计算

基于业务认知与数据特征，选定因变量与自变量之间的数学模型（如线性、非线性模型）；利用收集的数据，按照最小二乘等准则，计算模型中的未知系数（如回归系数）

03

模型比较与优化

通过统计分析方法（如决定系数 R^2 、剩余标准差 s 、F检验）对不同模型进行对比，筛选出拟合效果最优的模型

04

模型适用性检验

判断选定的模型是否适配当前数据集，验证模型的线性关系显著性（如相关系数 R 、F值检验）、参数估计可靠性等

05

模型应用

利用验证通过的模型，对因变量进行预测（如基于职工工资总额预测商品零售总额）或对变量间关系进行解释



YOUR
LOGO

02

整理一元线性回归 (J p126) 和多元线性回归 (J p136) 的基本数学模型



一元线性回归



模型定义

设 Y 是一个可观测的随机变量,它受到一个非随机变量因素 x 和随机误差 ε 影响。若 Y 与 x 有如下线性关系:

$$Y = \beta_0 + \beta_1 x + \varepsilon$$

经验回归方程

$$\begin{cases} \hat{\beta}_0 = \bar{y} - \bar{x}\hat{\beta}_1 \\ \hat{\beta}_1 = \frac{L_{xy}}{L_{xx}} \end{cases}$$

$$\text{其中, } \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, L_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2, L_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})。$$

于是就可以建立经验模型:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

模型参数说明

- Y : 可观测的随机因变量;
- x : 非随机自变量 (影响因素);
- β_0 、 β_1 : 固定未知的回归系数 (β_0 为截距项, β_1 为斜率项);
- ε : 随机误差项, 满足 $E(\varepsilon) = 0$ (均值为0)、 $\text{var}(\varepsilon) = \sigma^2 (\sigma > 0)$ (方差为常数)。

多元线性回归



模型定义

设 Y 是一个可观测的随机变量，它受到 $p(p > 0)$ 个非随机变量因素 $X_1, X_2, \dots, X_p$ 和随机误差 ε 的影响。若 Y 与 $X_1, X_2, \dots, X_p$ ，有如下线性关系：

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

矩阵形式

对于总体 $(X_1, X_2, \dots, X_p; Y)$ 的 n 组观测值 $(x_{i1}, x_{i2}, \dots, x_{ip}; y_i) (i=1, 2, \dots, n; n > p)$ ，应满足式：

$$\begin{cases} y_1 = \beta_0 + \beta_1 x_{11} + \beta_2 x_{12} + \dots + \beta_p x_{1p} + \varepsilon_1 \\ y_2 = \beta_0 + \beta_1 x_{21} + \beta_2 x_{22} + \dots + \beta_p x_{2p} + \varepsilon_2 \\ \dots \\ y_n = \beta_0 + \beta_1 x_{n1} + \beta_2 x_{n2} + \dots + \beta_p x_{np} + \varepsilon_n \end{cases}$$

其中 $\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n$ ，相互独立，且设 $\varepsilon_i \sim N(0, \sigma^2) (i=1, 2, \dots, n)$ ，记

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}, \quad \varepsilon = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

则模型可用矩阵形式表示为：

$$Y = X\beta + \varepsilon$$

模型参数说明

- Y ：可观测的随机因变量；
- $x_1, x_2, \dots, x_p$ ： p 个非随机自变量（多个影响因素）；
- $\beta_0, \beta_1, \dots, \beta_p$ ：固定未知的回归系数（ β_0 为截距项， $\beta_j (j=1, 2, \dots, p)$ 为各自变量对应的偏回归系数）；
- ε ：随机误差项，满足 $E(\varepsilon) = 0$ 、 $\text{var}(\varepsilon) = \sigma^2 (\sigma > 0)$ 。

经验回归方程

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_m x_m + \sum_{1 \leq j, k \leq m} \beta_{jk} x_j x_k + \varepsilon$$



03

整理Logistic回归的基本数学模型 (J p144) , 充分理解Logistic回归方法的含义(J p145), 能够讲清楚它的特点, 它与普通一元线性回归、多元线性回归的联系与区别



Logistic回归模型与含义



含义

Logistic回归本质是“分类导向的回归方法”，核心是通过建立自变量与因变量分类概率的映射关系，实现对分类结果的预测或影响因素分析。例如：

金融场景：基于客户收入、信用记录等自变量，预测客户“违约”（1类）或“不违约”（0类）的概率；

医疗场景：根据患者症状、检查指标，判断患者“患病”（1类）或“未患病”（0类）的可能性。

Logistic 回归模型的基本形式为：

$$p(Y=1|x_1, x_2, \dots, x_k) = \frac{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}{1 + \exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}$$

其中， $\beta_0, \beta_1, \dots, \beta_k$ 为类似于多元线性回归模型中的回归系数。

该式表示当变量为 $x_1, x_2, \dots, x_k$ 时，自变量 p 为 1 的概率。对该式进行对数变换，可得：

$$\ln \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

至此，我们就会发现，只要对因变量 p 按照 $\ln(p/(1-p))$ 的形式进行对数变换，就可以将 Logistic 回归问题转化为线性回归问题，此时就可以按照多元线性回归的方法很容易地得到回归参数。但很快又会发现，对于定性实践， p 的取值只有 0,1，这就导致 $\ln(p/(1-p))$ 形式失去意义。为此，在实际应用 Logistic 模型的过程中，常常不是直接对 p 进行回归，而是先定义一种单调连续的概率函数 π ，令

$$\pi = p(Y=1|x_1, x_2, \dots, x_k), \quad 0 < \pi < 1$$

有了这样的定义，Logistic 模型就可变形为：

$$\ln \frac{\pi}{1-\pi} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k, \quad 0 < \pi < 1$$

Logistic回归模型特点



因变量特性

因变量是分类变量（二分类或多分类，核心应用为二分类），而非连续变量

映射关系

通过Sigmoid函数实现非线性映射，但Logit变换后与自变量呈线性关系，属于“广义线性模型”

参数估计

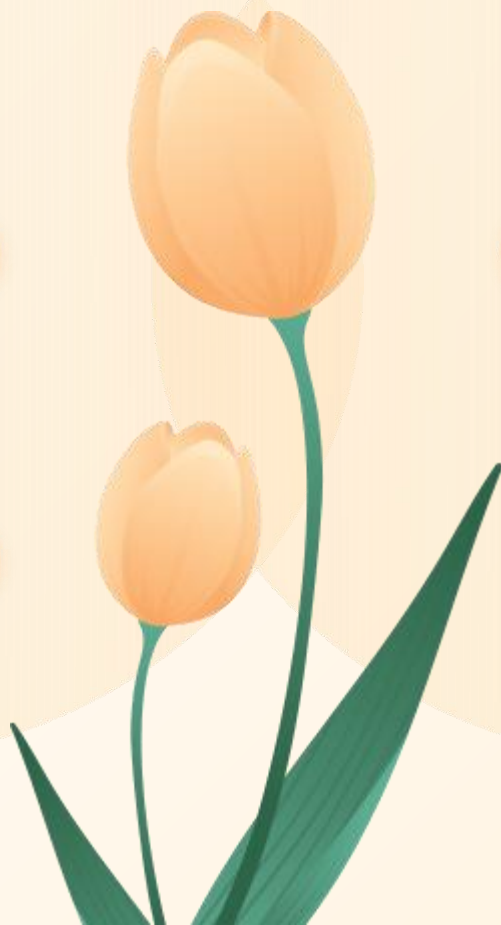
采用极大似然估计法求解回归系数，而非线性回归的最小二乘法

结果解释

输出结果为因变量取某一类别的概率，可直观反映事件发生的可能性，解释性较强

适用场景

适用于分类预测、影响因素筛选（如筛选影响疾病发生的关键指标），对数据分布要求较低



Logistic回归模型与普通线性回归的联系与区别



核心联系

核心联系

- 均属于回归分析框架，本质是研究自变量对因变量的影响关系
- 模型核心均包含线性组合部分 ($z = \beta_0 + \beta_1x_1 + \dots + \beta_px_p$)
回归系数均反映自变量的影响方向与强度
- 均可处理单自变量（一元）或多自变量（多元）的场景。

对比维度	普通线性回归 (一元/多元)	Logistic回归
因变量类型	连续变量	分类变量（二分类/多分类）
模型目标	预测因变量的具体连续取值	预测因变量取某一类别的概率，进而实现分类
函数关系	自变量与因变量呈直接线性关系	自变量与因变量的Logit概率呈线性关系，整体为非线性映射
误差项假设	误差项服从正态分布	无误差项正态分布假设，因变量服从二项分布（二分类场景）
参数估计方法	最小二乘法（最小化残差平方和）	极大似然估计法（最大化样本出现的概率）
输出结果	连续数值	概率值或分类结果
适用场景	预测连续变量取值	分类预测、影响因素分析



YOUR
LOGO

04

阅读《白话机器学习》，
第六章“回归”，并完
成读书笔记



第六章 “回归” 读书笔记



核心公式

模型	公式	核心说明
一元线性回归	$\hat{y} = \beta_0 + \beta_1 x$	单变量预测， β_1 为影响斜率
多元线性回归	$\hat{y} = \beta_0 + \sum_{i=1}^p \beta_i x_i$	多变量加权拟合， β_i 为变量权重
相关系数	$r = \beta^{\text{std}}$ (单变量)	取值 $[-1,1]$ ，量化关联方向与强度
梯度下降 (迭代)	$\beta \leftarrow \beta - \eta \cdot \nabla \text{Loss}$	学习率 η ，最小化误差找最优权重

避坑提醒

- 相关性 $\neq$ 因果：关联不代表推导（如养狗与房价正相关是收入导致）；
- 异常值敏感：少数异常点会严重扭曲趋势线；
- 多重共线性：变量间强相关会导致权重失真，需剔除或用正则化；
- 线性假设：仅适用于线性趋势，曲线需先变换数据。

关键指标

指标	含义	判定标准
标准化回归系数	消除单位，公平比较变量影响	数值越大，影响越强
相关系数r	单变量关联程度	$ r \rightarrow 1$ 强关联； $r=0$ 无关联
预测误差	模型预测精度	数值越小，模型越准

第三次作业

分类算法入门

从基础到实践

信管2302于景琪 202321054034

01

分类算法基础

基本逻辑与核心概念

分类算法是机器学习的核心分支，通过对已知类别的数据进行学习，构建决策边界以实现未知数据的类别预测。本章将深入解析其工作原理，掌握特征提取、模型训练与评估等关键环节的核心定义与逻辑框架。

什么是分类算法?

监督学习的核心分支：让计算机从已知“特征+标签”的数据中归纳规律，构建模型后，对未知类别的新数据进行演绎预测，实现从样本到泛化的智能判断。

逻辑本质与应用场景

生活中的广泛应用：涵盖垃圾邮件智能识别、医疗疾病辅助诊断、社交媒体情感倾向分析，以及计算机视觉中的图像内容分类等多个关键领域。



静谧之至，所谓绿
羊鸢绘

核心概念解析

01 特征 (Feature) 与 标签 (Label)

特征是描述样本属性的依据，如花瓣长度、邮件关键词；标签是样本所属的最终类别，如鸢尾花品种、是否为垃圾邮件。二者构成了模型学习的基础数据单元。

02 训练集 (Train) 与 测试集 (Test)

训练集如同模型学习的“练习题”，包含完整特征与标签用于模型拟合；测试集则是检验效果的“期末考试”，通过未知标签的样本验证模型泛化能力，确保模型具备真实场景的适用性。



03 基础评估：准确率与精确率

准确率是分类正确样本占总样本的比例，反映整体判断能力；精确率聚焦“预测为正例”的样本中真正正例的占比，衡量模型判断正例的精准度，适用于避免误报的场景。

04 关键维度：召回率与综合考量

召回率是实际正例中被成功预测的比例，衡量模型对正例的捕捉能力，适用于避免漏报的场景。在实际业务中，需根据场景需求平衡精确率与召回率，构建更合理的评估体系。

02

KNN算法详解

深入解析K近邻算法的核心思想与执行步骤，结合经典机器学习读物的读书笔记，拆解算法原理与应用场景。

KNN算法的核心思想



特点：“惰性学习”

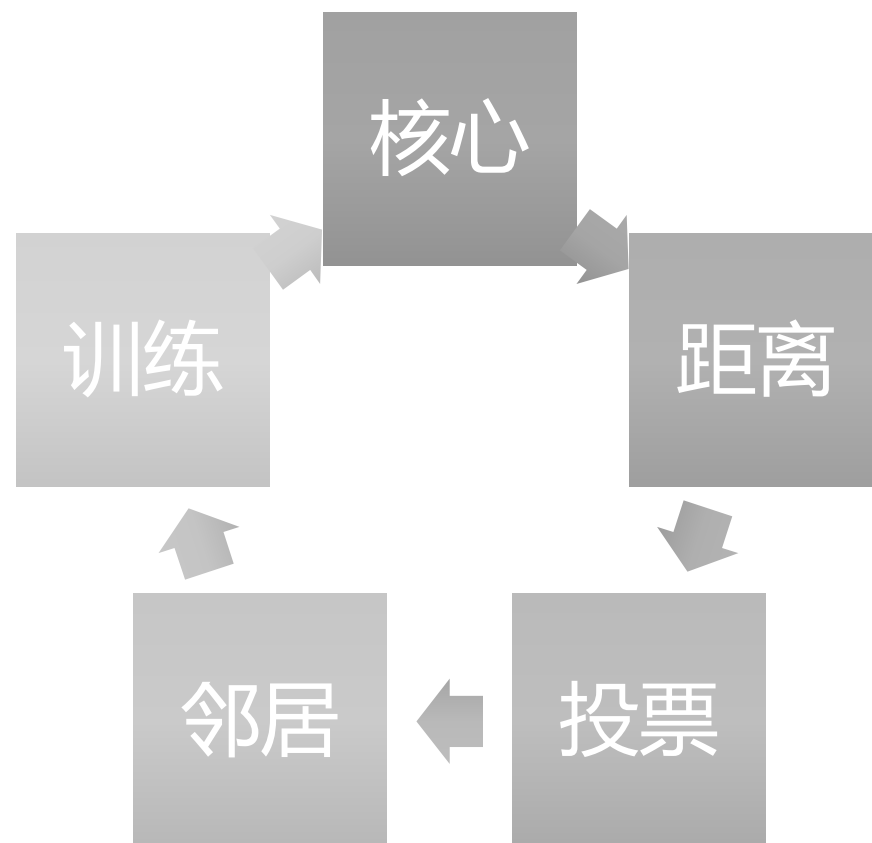
KNN无显式训练过程，训练阶段仅存储数据集，所有计算均在预测阶段发生。如同平时不复习、考试才翻书找答案的学生，虽简化了训练，但预测时计算成本会随数据量增加而上升。

核心：“物以类聚”

KNN算法的逻辑非常直观，一个样本的类别由其周围离它最近的K个邻居的类别决定，遵循“少数服从多数”的投票原则。就像判断一个人的品性看其交往的朋友，特征空间中近邻的样本往往具有相似属性。



KNN算法核心步骤



训练阶段： 收集带标签样本，对特征做归一化处理以消除量纲影响，最后存储数据备用。

预测阶段： 计算新样本与训练集的距离，选取最近的K个邻居，通过“少数服从多数”的投票机制确定预测类别，是典型的懒惰学习算法。

ML · K-NEAREST NEIGHBORS



03

《白话机器学习》读书笔记

本书用生动通俗的语言拆解了KNN与决策树算法的核心逻辑，从生活化的案例出发，将复杂的距离计算与分支判断转化为直观易懂的思维模型，帮助我们跳出公式推导，理解算法背后的决策本质与应用场景。



第七章 KNN：看看邻居们都投谁

■ 核心逻辑：物以类聚，人以群分

判断一个样本的类别，就看它周围最近的邻居们都是什么类别。如果邻居大多属于某类，那它大概率也属于该类，就像通过观察“朋友圈”来判断一个人的特质。

■ 关键参数：k值的双刃剑效应

k值过小：如同偏听少数密友，易受个别“噪声点”干扰，导致模型过拟合，泛化能力差。

k值过大：如同盲从大众意见，会忽略样本的局部特征，导致模型欠拟合，决策过于模糊。

■ 模型总结：简单但有局限

优势是原理直观、无需训练；劣势是预测速度慢，且对数据的尺度和异常值非常敏感，需先做标准化处理。



第九章 决策树：猜谜式推理

核心逻辑：像玩“20个问题”游戏。通过不断提问“是不是”来缩小范围，利用信息增益或基尼系数选择最优特征，让每一次划分都能最大化数据的“纯度”，快速逼近答案。

LOGIC . SPLIT . OPTIMIZE

生长与剪枝：平衡复杂度与泛化。决策树会自然生长至叶节点，但过度生长易导致过拟合。需通过“剪枝”剔除冗余分支，简化模型结构，从而提升对未知数据的适应能力。

静谧之至，
所谓绿
羊鸢绘



04 算法比较与实践

其他算法探索与案例分析

在掌握KNN与决策树基础上，进一步深入探索更多经典分类算法，结合真实业务场景进行实战应用，从原理、效率、适用边界等维度对各类算法进行全面对比分析，沉淀算法选型的核心逻辑。

其他分类算法核心思想

朴素贝叶斯

基于贝叶斯定理和特征条件独立假设，通过计算样本属于各类别的后验概率实现分类。该算法计算效率高，尤其适合处理文本、邮件等稀疏的高维数据场景。

神经网络

模仿人脑神经元的连接结构，构建多层网络对输入数据进行逐层抽象与特征变换，最终输出分类结果。作为深度学习的基础，广泛应用于图像识别、NLP等复杂任务。

决策树

通过构建一系列“if-else”的逻辑判断规则，将数据集逐步划分到不同的类别中，形成树状的决策结构。其优势在于决策过程直观透明，结果易于解释和理解。



朴素贝叶斯：垃圾邮件识别



01 数据准备 | 02 特征提取

首先收集已人工标记的垃圾邮件与正常邮件作为数据集。接着构建全局词汇表，将每封邮件转化为0-1特征向量，向量维度对应词汇表，元素值表示对应词汇是否在邮件中出现，完成文本数据的数字化转换。

03 模型训练 | 04 概率预测

训练阶段需计算先验概率（垃圾邮件占比）与条件概率（词汇在两类邮件中的出现频率），构建概率模型。预测时，将新邮件向量化后代入模型，分别计算其属于垃圾邮件和正常邮件的后验概率，依据“极大似然”原则判定类别。

05

算法比较

KNN vs. 决策树 vs. 朴素贝叶斯

以经典的鸢尾花分类问题为切入点，深度剖析三种基础分类算法的核心原理、优缺点及适用场景，为算法选型提供清晰依据。

算法比较：KNN vs. 决策树 vs. 朴素贝叶斯

KNN (K近邻)：直观但敏感

优点是简单直观且无分布假设；缺点是预测速度慢，对特征缩放敏感。适用于数据量较小的场景，如基础推荐系统的初步匹配。

朴素贝叶斯：高效的文本处理专家

训练与预测速度极快，适合高维稀疏数据；但特征独立的“朴素”假设在现实中常不成立。是文本分类、垃圾邮件过滤的经典首选算法。

选型核心：适配场景而非最优

没有绝对完美的算法，只有最适配的方案。需根据数据特征（如维度、量纲）、业务需求（如是否需要解释）以及性能要求综合权衡决策。

决策树：可解释性强但易过拟合

规则清晰易懂，可解释性极强；但容易过度学习局部数据导致过拟合，稳定性较差。常用于需要明确决策逻辑的领域，如医疗诊断与风控规则生成。



总结与收获



01

夯实理论基础：系统理解分类算法的基本范式，厘清了混淆矩阵、准确率、召回率等核心评估指标的内涵，建立了对机器学习分类任务的完整认知框架。

02

深入核心算法：掌握KNN的近邻投票机制与决策树的信息增益分裂逻辑，剖析了两者在不同数据场景下的优缺点，学会通过调参优化模型的泛化能力。

03

拓宽技术视野：探索了朴素贝叶斯的概率推理、神经网络的层级映射等多元算法，掌握了算法对比选型的核心维度，能够依据数据特征与业务目标合理选择技术方案。

通过对比不同算法的适用边界，深刻体会到“没有最好的算法，只有最适合场景的算法”这一机器学习核心原则，提升了工程落地的思维能力。

第四次作业

聚类算法入门

物以类聚，人以群分 —— 探索无监督学习的核心规律

信管2302于景琪 202321054034

01

聚类基础

深入解析“簇”的核心定义，探究数据点间相似性的科学度量方式，为后续聚类算法的学习奠定坚实的理论基础。

什么是簇 (Cluster)?

核心定义：簇是一组数据对象的集合，满足“簇内相似性高，簇间相异性高”。核心是让簇内数据点尽可能彼此相似，同时确保不同簇之间有明显的界限，实现数据的自然分组。

核心思想与关键特征

关键要素：簇存在代表中心位置的“质心”；聚类属于无监督学习，无需预设标签，完全依靠数据自身的特征相似度来自动形成簇结构，挖掘数据内在规律。

静谧之至，
所谓绿
羊鸢绘



距离度量的两种基本方法

01. 欧氏距离 (Euclidean Distance)

定义为两点间的“直线距离”，用于衡量数值上的绝对差异。公式为 $d = \sqrt{\sum (x_i - y_i)^2}$ ，是最直观的空间距离度量方式。

欧氏距离：适用与局限

适用于物理空间定位、多维数值比较等场景。但它对量纲高度敏感，计算前通常需要对数据进行标准化处理，否则会受量纲影响产生偏差。

02. 余弦距离 (Cosine Distance)

衡量向量方向差异，公式为 $d = 1 - \cos(\theta)$ 。不受向量长度影响，适合文本相似度、用户兴趣分析等高维稀疏数据场景，聚焦方向而非大小。

核心差异：聚焦方向而非数值

与欧氏距离关注数值绝对值不同，余弦距离更关注两个向量的夹角。即使数值规模不同，只要方向一致，余弦相似度就高，是NLP领域的核心度量。



02

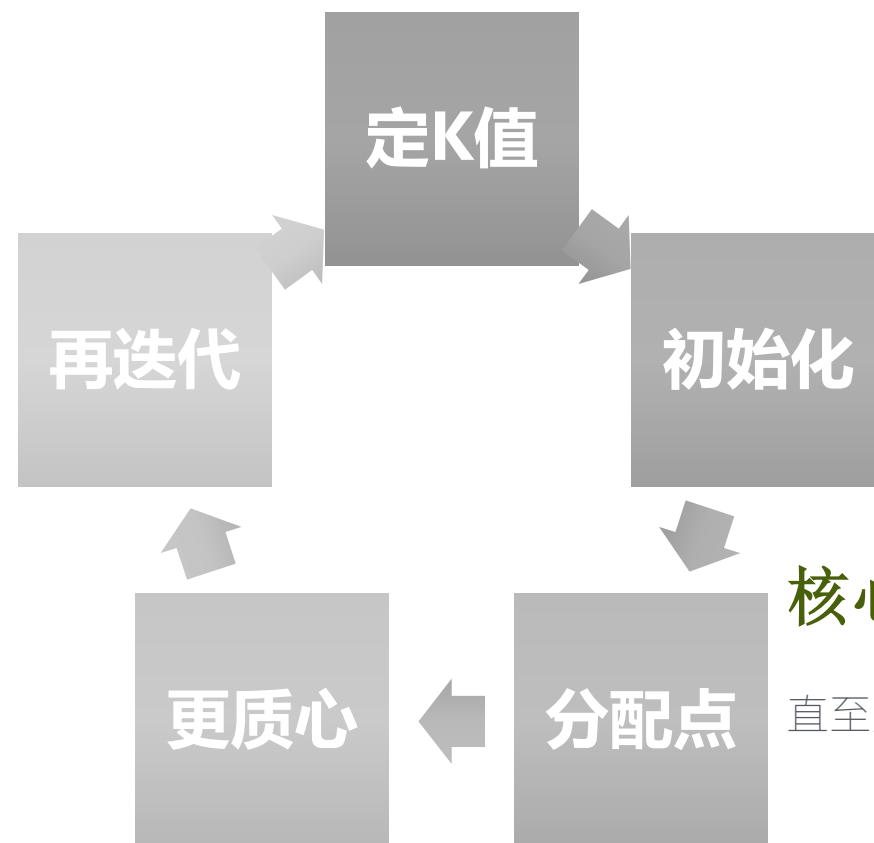
K-means算法

深入解析经典聚类算法的核心原理与执行步骤，结合《白话机器学习》读书笔记，梳理算法应用的关键要点与实践思考。



K-means算法原理与步骤

核心原理：基于“分配-更新”的迭代优化循环，通过计算样本间距离，将数据划分为K个簇，使簇内相似度高、簇间相似度低，最终收敛至稳定的质心位置。



核心逻辑：迭代收敛·聚类划分
直至质心不再变化或达到最大迭代次数终止循环。



K-means 聚类读书笔记



核心比喻：像评论员给球队分档

算法过程如同足球赛事分级：先随机指定几个“标杆球队”作为初始质心，将所有球队归到距离最近的标杆组；随后根据分组结果重新计算更合理的标杆位置，反复迭代直到分组稳定不再变化，最终形成清晰的梯队划分。

关键技巧：肘部法则选K值

尝试不同K值并绘制簇内总距离曲线，曲线由陡变缓的“肘部”即为最佳分组数。同时需注意，初始质心的随机选择可能影响结果，实践中需多次初始化规避局部最优陷阱。



03

层次聚类 (HC)

算法过程与结果解读：层次聚类（Hierarchical Clustering）不直接生成最终聚类簇，而是构建数据的层级嵌套结构。它通过不断合并相似簇或分裂差异簇，形成树状的聚类谱系图（Dendrogram），能清晰展示数据从整体到局部的聚合关系，为分析数据的内在层次提供了直观视角。

HC算法过程与树状图解读

01. 凝聚式HC核心过程

算法始于“初始化”，视每个数据点为独立簇；随后进入“迭代合并”，持续寻找距离最近的两个簇进行融合；最终“重复操作”直至所有数据点合并为单一簇，完成层级构建。

02. 算法关键特性

该过程具有“自底向上”的层级特征，无需预先指定簇的数量。合并的依据（距离/相似度）决定了最终的聚类形态，常见的距离度量包括欧氏距离、曼哈顿距离等。



03. 树状图(Dendrogram)坐标轴解读

横轴代表簇间的“距离或相似度”，数值越大表示簇的差异越明显；纵轴则对应原始数据点或合并后的簇。树状图的分支结构直观展示了数据从细粒度到粗粒度的聚合路径。

04. 确定聚类数量：以5类为例

在树状图上绘制一条水平切割线，使其与图中的垂直分支恰好相交于5个位置。交点下方的所有数据点即构成独立的5个簇，该方法是层级聚类中确定类别数的最直观手段。

总结与收获

01. 夯实基础认知：深入理解了“簇”的核心概念，熟练掌握了距离度量的两种关键方法，为聚类分析构建了坚实的理论基石。

02. 吃透核心算法：系统掌握了K-means算法的迭代优化逻辑与层次聚类的聚合/分裂步骤，明晰了两类经典算法的适用场景与核心差异。

03. 掌握结果解读：学会利用树状图（谱系图）的层级结构特征，灵活判断与确定最优聚类数量，让数据的内在结构得以可视化呈现。

第五讲 关联规则与 Apriori算法

信管2302 于景琪

202321054034

1.整理资料：有关关联选股的背景知识

1.关联选股是什么

利用**关联规则挖掘**，找出**行业之间、股票之间、股票与行业之间**的联动规律，例如：
某行业上涨时，另一行业大概率也上涨；某几只股票经常同涨同跌。
用这种**共现规律**构建投资组合、择时或分散风险。

2.核心思想

股票/行业的涨跌可以看作“事务”，上涨的股票/行业看作“被购买的商品”，
用**Apriori 算法**找出**频繁共涨**的组合 → 形成强关联规则 → 指导选股。

3.应用价值

发现板块轮动规律；构建联动性强的股票组合；辅助预测走势、降低持仓相关性风险

2.分析数据（清楚说明矩阵行与列的含义）

行 = 时间片段（事务），列 = 行业 / 股票（项），值 = 是否上涨。

在关联选股的数据表示中，通常使用**0-1 矩阵**表示：

- **行（Row）**：代表一个时间窗口/一次交易事务

例如：某一天、某一周的市场表现。

- **列（Column）**：代表一个行业/一只股票（即“项” Item）

- **矩阵元素的值：**

- **1**：该行业/股票在当期**上涨（或满足条件）**

- **0**：该行业/股票在当期**未上涨**



3.运行程序

jupyter 生成表格数据 Last Checkpoint: 39 seconds ago

File Edit View Run Kernel Settings Help

+ ✂ 📄 ▶ ■ ↺ ▶▶ Code

```
[1]: import csv
import random

# 生成商品池 (示例商品名称)
products = [
    "智能手机", "蓝牙耳机", "机械键盘", "智能手表", "平板电脑",
    "无线鼠标", "笔记本电脑", "显示器", "移动电源", "摄像头",
    "游戏手柄", "投影仪", "打印机", "路由器", "硬盘",
    "电动牙刷", "吹风机", "空气炸锅", "咖啡机", "吸尘器",
    "加湿器", "电风扇", "电饭煲", "微波炉", "烤箱"
]

# 生成数据行
data = []
for i in range(1, 101):
    selected = random.sample(products, 3) # 确保同一行三个不重复
    data.append([i, selected[0], selected[1], selected[2]])

# 写入CSV文件
with open('products.csv', 'w', newline='', encoding='utf-8-sig') as f:
    writer = csv.writer(f)
    writer.writerow(["序号", "商品1", "商品2", "商品3"])
    writer.writerows(data)

print("CSV文件已生成: products.csv")
```

CSV文件已生成: products.csv

jupyter products.csv Last Checkpoint: 1 minute ago

File Edit View Settings Help

Delimiter: ,

	序号	商品1	商品2	商品3
1	1	投影仪	机械键盘	加湿器
2	2	笔记本电脑	摄像头	移动电源
3	3	咖啡机	机械键盘	无线鼠标
4	4	无线鼠标	微波炉	加湿器
5	5	笔记本电脑	吸尘器	打印机
6	6	吸尘器	加湿器	无线鼠标
7	7	加湿器	电风扇	蓝牙耳机
8	8	空气炸锅	智能手机	电动牙刷
9	9	投影仪	蓝牙耳机	吸尘器
10	10	移动电源	笔记本电脑	空气炸锅
11	11	平板电脑	硬盘	加湿器
12	12	投影仪	吹风机	电动牙刷
13	13	移动电源	平板电脑	智能手表
14	14	微波炉	咖啡机	机械键盘
15	15	打印机	吹风机	加湿器
16	16	显示器	硬盘	吸尘器
17	17	微波炉	智能手机	烤箱
18	18	显示器	机械键盘	电风扇
19	19	智能手表	机械键盘	平板电脑
20	20	电风扇	吸尘器	电饭煲
21	21	吸尘器	烤箱	电动牙刷
22	22	显示器	智能手机	蓝牙耳机
23	23	路由器	打印机	显示器

3.运行程序

jupyter 使用apriori构建商品知识图谱 Last Checkp

File Edit View Run Kernel Settings Help

+ ✂ 📄 📄 ▶ ■ ↻ ▶▶ Code ▾

[12]: ##### 步骤1: 安装依赖库 (如果尚未安装)

```
!pip install mlxtend
!pip install networkx
```

Requirement already satisfied: mlxtend in d:\sb\li
Requirement already satisfied: scipy>=1.16.3 in d:
Requirement already satisfied: numpy>=2.3.5 in d:\
Requirement already satisfied: pandas>=2.3.3 in d:
Requirement already satisfied: scikit-learn>=1.8.0
Requirement already satisfied: matplotlib>=3.10.8
Requirement already satisfied: joblib>=1.5.2 in d:
Requirement already satisfied: contourpy>=1.0.1 in
Requirement already satisfied: cycler>=0.10 in d:\
Requirement already satisfied: fonttools>=4.22.0 i
Requirement already satisfied: kiwisolver>=1.3.1 in d:\sb\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.4.8)
Requirement already satisfied: packaging>=20.0 in d:\sb\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (24.2)
Requirement already satisfied: pillow>=8 in d:\sb\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (11.1.0)
Requirement already satisfied: pyparsing>=3 in d:\sb\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in d:\sb\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (2.9.0.po
Requirement already satisfied: tzdata in d:\sb\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: six>=1.5 in d:\sb\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.10.8->mlxtend)
Requirement already satisfied: threadpoolctl>=3.2.0 in d:\sb\lib\site-packages (from scikit-learn>=1.8.0->mlxtend) (3.5.0)

WARNING: Ignoring invalid distribution ~cipy (D:\sb\Lib\site-packages)
WARNING: Ignoring invalid distribution ~cipy (D:\sb\Lib\site-packages)
WARNING: Ignoring invalid distribution ~cipy (D:\sb\Lib\site-packages)
Requirement already satisfied: networkx in d:\sb\lib\site-packages (3.4.2)

WARNING: Ignoring invalid distribution ~cipy (D:\sb\Lib\site-packages)
WARNING: Ignoring invalid distribution ~cipy (D:\sb\Lib\site-packages)
WARNING: Ignoring invalid distribution ~cipy (D:\sb\Lib\site-packages)

[13]: ##### 步骤2: 导入所需库

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
```

[16]: trans_df

[16]:

	加湿器	吸尘器	吹风机	咖啡机	平板电脑	微波炉	打印机	投影仪	摄像头	无线鼠标	...	烤箱	电动牙刷	电风扇	电饭煲	硬盘	移动电源	空气炸锅	笔记本电脑	蓝牙耳机	路由器
0	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	False	False	True	False	False	False
1	True	False	False	False	False	True	False	False	False	False	...	False	True	False	False	False	False	False	False	False	False
2	False	False	True	False	False	False	False	False	False	False	...	False	False	True	False	False	False	False	False	True	False
3	False	False	False	True	False	False	False	False	False	False	...	False	True	True	False	False	False	False	False	False	False
4	False	True	False	True	False	False	False	False	False	False	...	False	False	False	False	False	False	True	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
95	True	False	False	False	False	False	False	F													
96	False	False	False	False	False	False	True	F													
97	False	False	False	False	False	True	False	F													
98	False	False	False	False	False	True	False														
99	False	False	False	False	False	False	False	F													

100 rows × 25 columns

计算实际出现次数

```
frequent_itemsets['count'] = (frequent_itemsets['support'] * len(transactions
qualified = frequent_itemsets[(frequent_itemsets['length'] == 2) & (frequent_
```

qualified

	support	itemsets	length	count
25	0.03	[加湿器, 平板电脑]	2	3
26	0.02	[加湿器, 摄像头]	2	2
27	0.03	[加湿器, 电动牙刷]	2	3
28	0.03	[加湿器, 硬盘]	2	3
29	0.03	[加湿器, 笔记本电脑]	2	3
...	...	...	...	...
96	0.03	[烤箱, 路由器]	2	3
97	0.02	[电风扇, 蓝牙耳机]	2	2
98	0.02	[电风扇, 路由器]	2	2
99	0.03	[电饭煲, 路由器]	2	3
100	0.02	[空气炸锅, 路由器]	2	2

76 rows × 4 columns

3.运行程序

jupyter 使用apriori构建商品知识图谱 Last Checkpoint

File Edit View Run Kernel Settings Help

Code

```
# 设置布局
pos = nx.spring_layout(G, k=0.5)

# 绘制节点
nx.draw_networkx_nodes(G, pos, node_size=1500, node_color='lightblue')

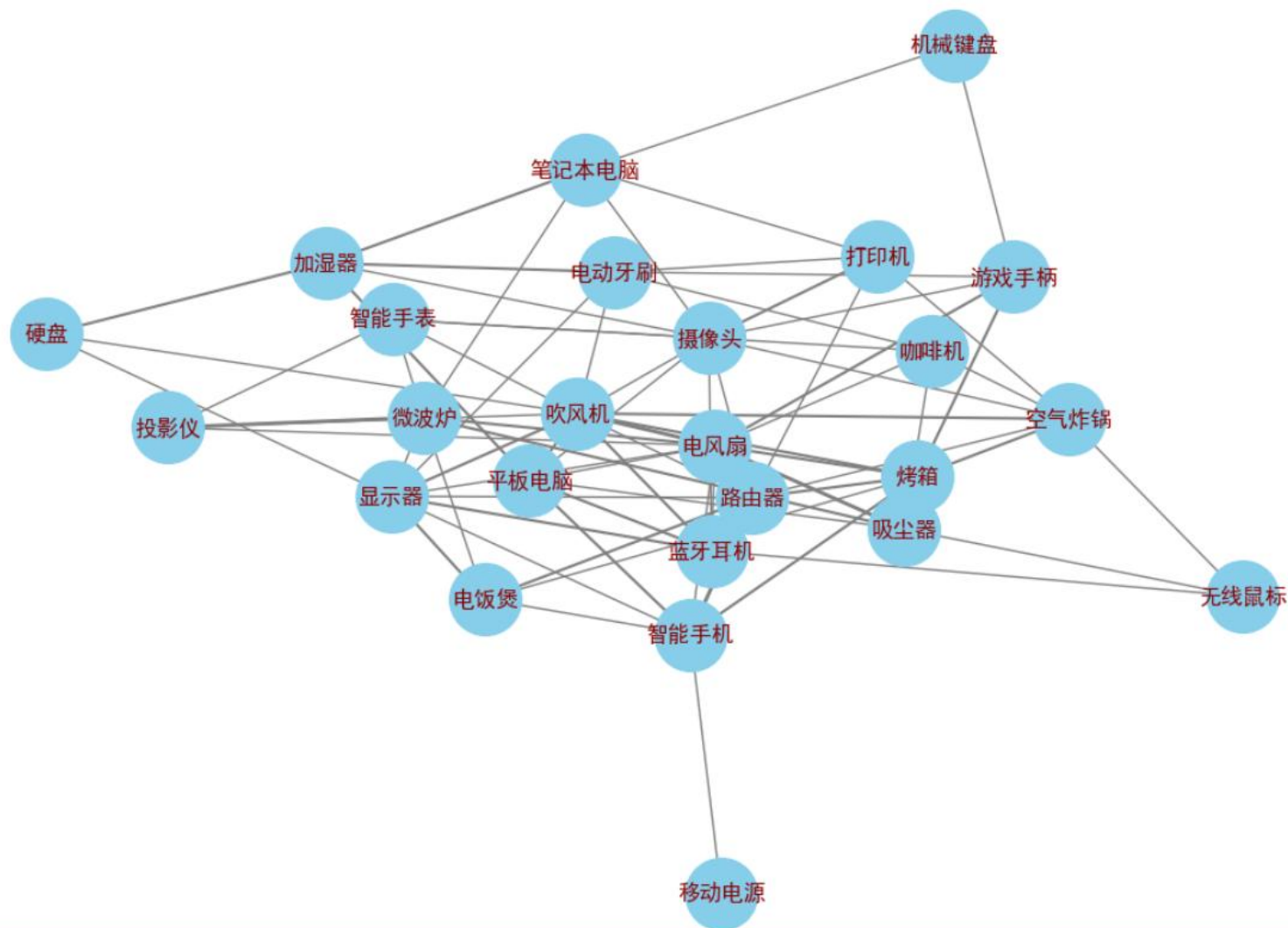
# 绘制边
edges = G.edges(data=True)
nx.draw_networkx_edges(G, pos, edgelist=edges,
                        width=[d['weight']*0.5 for (u,v,d) in edges],
                        edge_color='gray')

# 绘制标签 (删掉了错误的Songti SC, 用全局配置的字体)
nx.draw_networkx_labels(G, pos, font_size=12,
                        font_color='darkred')

# 显示图表 (删掉了错误的Arial Unicode MS, 用全局配置的字体)
plt.title("商品关联分析",
        fontdict={
            'fontsize': 16,
            'fontweight': 'bold',
            'color': 'navy'
        })

plt.axis('off')
```

商品关联分析



4. 分析结果，思考教材分析过程的含义（并指出教材中的错误）

1. 结果含义解释

- **支持度高**：两个行业 / 股票经常一起上涨，联动性稳定。
- **置信度高**：一个上涨大概率带动另一个上涨，预测性强。
- **强关联规则**：可用于配对交易、板块配置、趋势预判。

2. 教材中的错误

- 表 6-1 事实错误：事务 5 不包含网球拍，却被计入同时购买网球拍和网球，统计错误。
- P114 支持度计数错误：{I2,I4} 支持度计数应为 2，教材写成 4。
- P123 前后矛盾：行业关联选股中 {银行，券商，钢铁} 支持度表格为 3/10，文字写成 4/10。



第六讲

线性回归实践与评估

模型评估与优化核心要点

深入解析数据预处理规范、回归模型性能指标体系，掌握通过正则化与交叉验证提升模型泛化能力的关键实践路径。

信管2302 于景琪 202321054034

核心概念：维度与评估

输入特征X的维度规范

标准形状需为(样本数量, 特征数量)的二维数组。单特征场景下，X必须是列向量形式(如25,1)，而非行向量(1,25)，这是矩阵运算的基础要求。

模型评估：决定系数 R^2

R^2 衡量模型拟合优度，值越接近1说明拟合效果越好。例如 $R^2=0.8$ 代表因变量80%的变异可由模型解释，是判断模型解释能力的核心指标。

维度错误的关键影响

错误的维度输入会直接导致矩阵乘法运算失败，引发程序报错；即使部分框架容错运行，也会造成模型训练逻辑错误，最终产出无意义的预测结果。

```
[4]: X = dataset.iloc[:,0:1].values #必须是列向量
```

```
[5]: X
```

```
[5]: array([[2.5],  
          [5.1],  
          [3.2],  
          [8.5],  
          [3.5],  
          [1.5],  
          [9.2],  
          [5.5],  
          [8.3],  
          [2.7],  
          [7.7],  
          [5.9],  
          [4.5],  
          [3.3],  
          [1.1],  
          [8.9],  
          [2.5],  
          [1.9]])
```

```
[6]: X.shape
```

```
[6]: (25, 1)
```

模型实践

01. 获取模型核心参数

利用训练集(X_train, Y_train)完成模型拟合后，可直接提取关键参数：通过 regressor.coef_ 获取系数 b1，通过 regressor.intercept_ 获取截距 b0，最终形成预测方程，例如： $y = 2.02 + 9.91 * x_0$ 。

02. 参数的业务解读

系数反映了自变量对因变量的边际影响，截距则是基准值。在实际业务中，需结合领域知识验证参数的合理性，确保模型输出符合业务逻辑，而非单纯依赖数值结果。

03. 最优数据集划分比例

测试集 R^2 会随比例变化呈现先升后降趋势，实验验证测试集比例控制在 20%~30% 区间时，模型的泛化能力达到最佳平衡，既能保证训练数据充足，又能有效评估模型的未知数据适应性。

```
b1 = regressor.coef_ # 系数
b1

array([9.91065648])

b0 = regressor.intercept_ #截距 y = 2 + 9.9x
b0

np.float64(2.018160041434683)
```

04. 警惕过拟合信号

若训练集 R^2 远高于测试集 R^2 ，是典型的过拟合表现。此时需调整数据集比例、增加数据量或引入正则化手段，避免模型过度学习训练数据中的噪声，确保模型具备稳定的鲁棒性。

案例分析：模型评估与可视化

- 3 评价结果的好坏 R^2

```
R2_train = regressor.score(X_train,Y_train)  
R2_train
```

0.9515510725211552

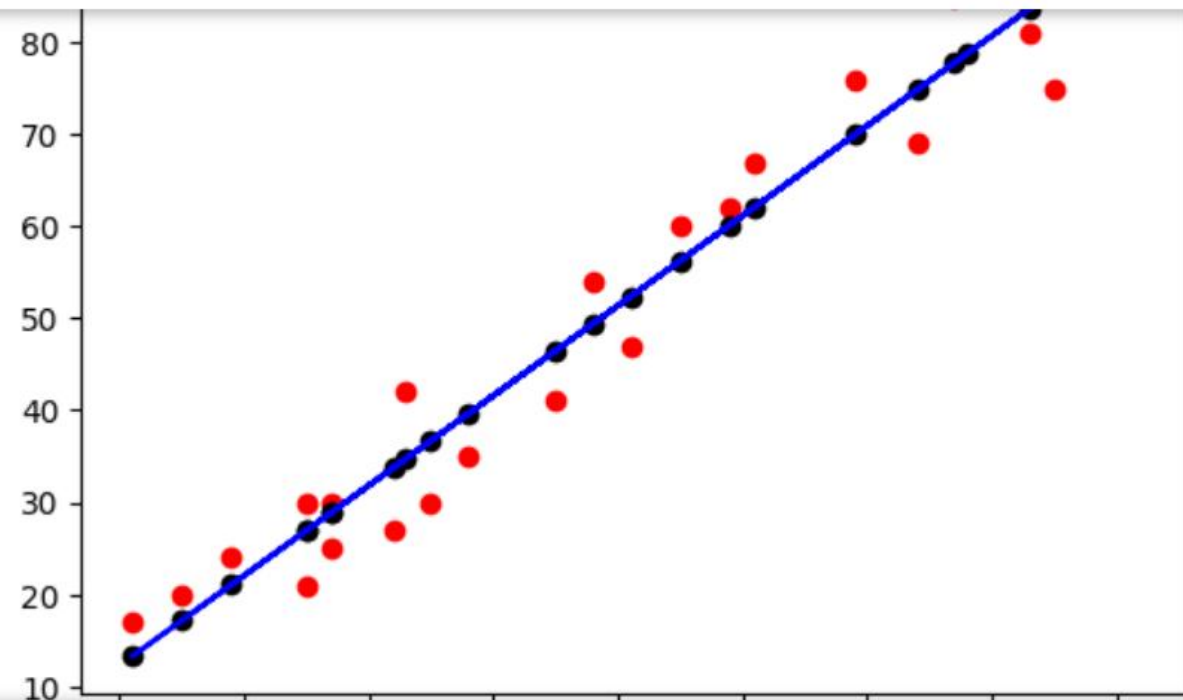
```
R2_test = regressor.score(X_test,Y_test)  
R2_test
```

0.9454906892105355

01. 数据划分与评估指标

采用train_test_split函数完成数据集的科学划分。模型评估显示：训练集 R^2 值达0.9516，测试集 R^2 值为0.9455，两项核心指标均处于高位区间。

```
[18]: plt.scatter(X, Y, color = 'red')  
plt.scatter(X, Y_pred0, color = 'black')  
plt.plot(X,regressor.predict(X), color= 'blue')
```



02. 拟合结论与泛化能力

训练集与测试集的 R^2 值高度接近且数值优异，表明模型对数据特征捕捉充分，拟合效果极佳，同时具备较强的泛化能力，能稳定适配新数据场景。

第3步 预测结果

```
Y_pred = regressor.predict(X_test) # .predict # 用熟模型去预测
```

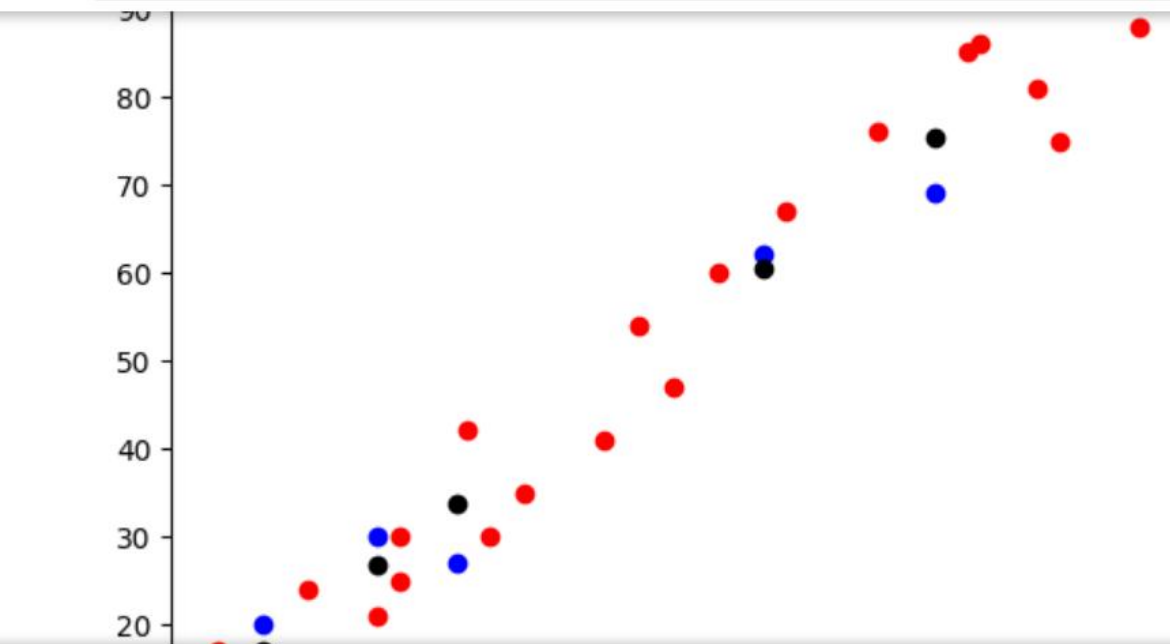
Y_pred

```
array([16.88414476, 33.73226078, 75.357018 , 26.79480124, 60.49103328])
```

第4步 可视化

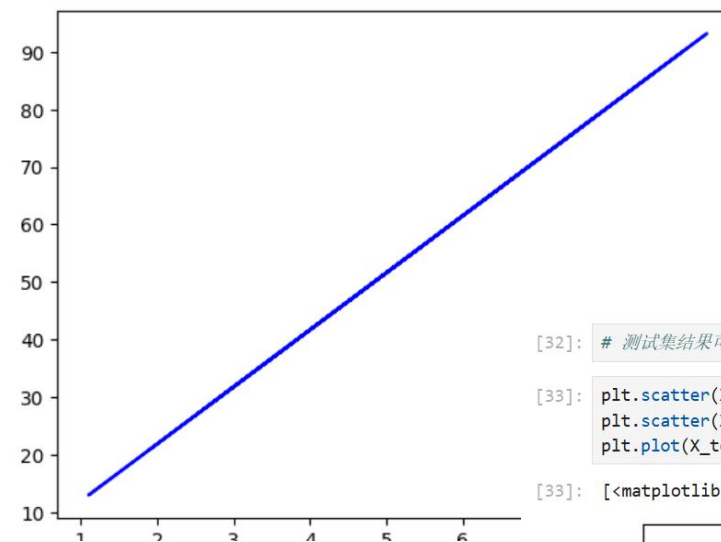
```
[25]: # 训练集结果可视化  
# 特别注意 scatter和plot的区别。
```

```
[26]: plt.scatter(X_train, Y_train, color = 'red')  
plt.scatter(X_test, Y_test, color = 'blue')  
plt.scatter(X_test, Y_pred, color = 'black')
```



```
27]: plt.plot(X_train, regressor.predict(X_train), color = 'blue')
```

```
27]: [ <matplotlib.lines.Line2D at 0x1b535d7bc50>]
```



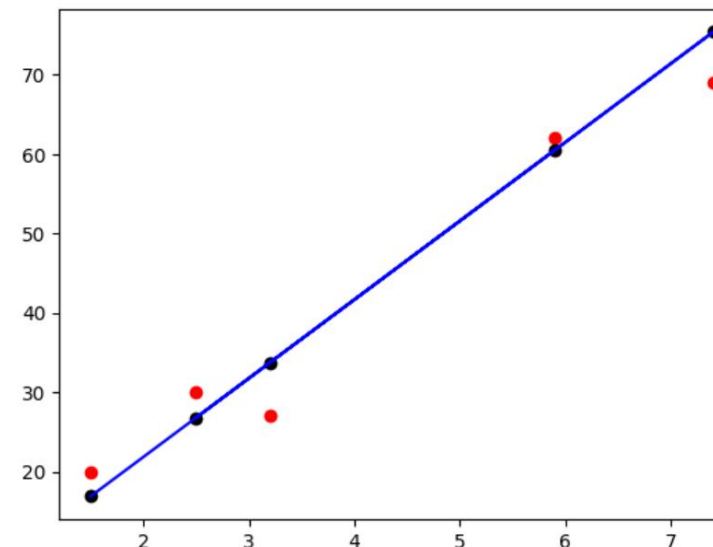
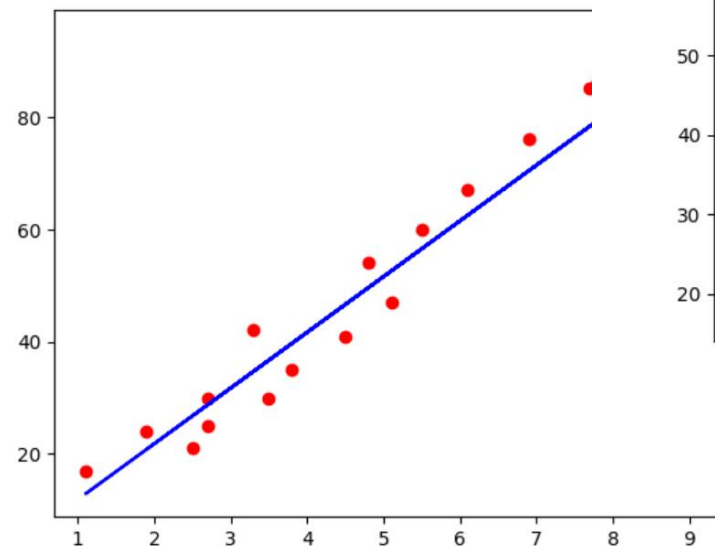
```
[32]: # 测试集结果可视化
```

```
[33]: plt.scatter(X_test, Y_test, color = 'red')  
plt.scatter(X_test, Y_pred, color = 'black')  
plt.plot(X_test, Y_pred, color = 'blue')
```

```
[33]: [ <matplotlib.lines.Line2D at 0x1b535f84050>]
```

```
: plt.show()
```

```
: plt.scatter(X_train, Y_train, color = 'red')  
plt.plot(X_train, Y_pred_train, color = 'blue')  
plt.show()
```



关键结论总结

线性回归模型评估核心要点回顾

01. 数据格式是基石：必须确保输入特征 X 为 (样本数, 特征数) 的二维数组结构，这是模型运行的基础前提。
02. R^2 是核心评估指标： R^2 值越接近 1，代表模型对数据的解释能力越强，是衡量拟合效果的关键。
03. 警惕过拟合风险：若训练集 R^2 远高于测试集 R^2 ，说明模型泛化能力差，需及时调整模型复杂度。
04. 科学划分数据集：推荐按 70%~80% 为训练集、20%~30% 为测试集的比例划分，保障模型验证的有效性。

第七次

多元线性回归作业

信管2302 于景琪

202321054034

1.整理资料：多元线性回归背景知识

1.核心概念解析

多元线性回归聚焦研究一个因变量与多个自变量之间的线性依存关系，核心是通过数据拟合构建最优“超平面”，以此实现对因变量的精准解释与预测。其标准模型形式为： $Y = b_0 + b_1X_1 + \dots + b_nX_n + \varepsilon$ ，其中Y为因变量，X为自变量，b为回归系数， ε 为随机误差项。

2.实际应用价值

模型具备三大核心用途：一是**预测分析**，基于自变量变化推导因变量未来取值；二是**因素归因**，识别对因变量有显著影响的关键因子并量化其影响程度；三是**趋势研判**，厘清多变量间的相互作用逻辑与动态变化趋势。

3.本作业核心：分类变量处理

当自变量包含城市、类别标签等分类数据时，无法直接代入数值模型计算，需采用**独热编码 (One-Hot Encoding)** 将其转化为二进制虚拟特征，以此消除分类间的虚假顺序关系，确保模型能正确识别与解析类别属性的实际意义。

2.分析数据：数据集与可视化

数据集：50_Startups.csv，涵盖50家创业公司的多维经营数据，聚焦利润影响因子分析。

核心变量构成解析：

- **自变量 (X)**：包含研发支出(R&D Spend)、行政管理支出(Administration)、营销支出(Marketing Spend)及分类变量州(State)，覆盖投入与地域维度。
- **因变量 (Y)**：以公司年利润(Profit)为核心分析目标，衡量各项投入与地域因素的最终产出效果。

地域利润分布可视化洞察：

通过散点图探索州(State)与利润的关联，发现纽约、加州、佛罗里达三个州的利润分布区间高度重叠，未呈现出某一州的利润具有绝对优势或劣势的特征，地域并非利润差异的显著单一影响因素。

图表：三州创业公司利润散点分布。Y轴为利润值，X轴为州名，可见数据点分布趋同。

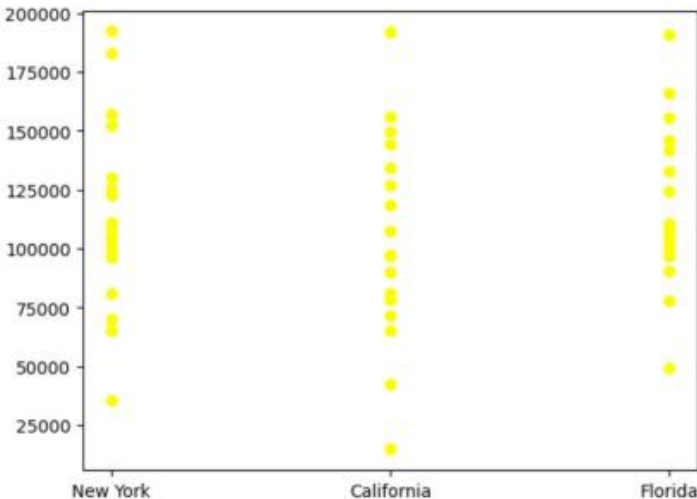
导入数据集

```
[2]: dataset = pd.read_csv('50_Startups.csv')
      dataset.head()
```

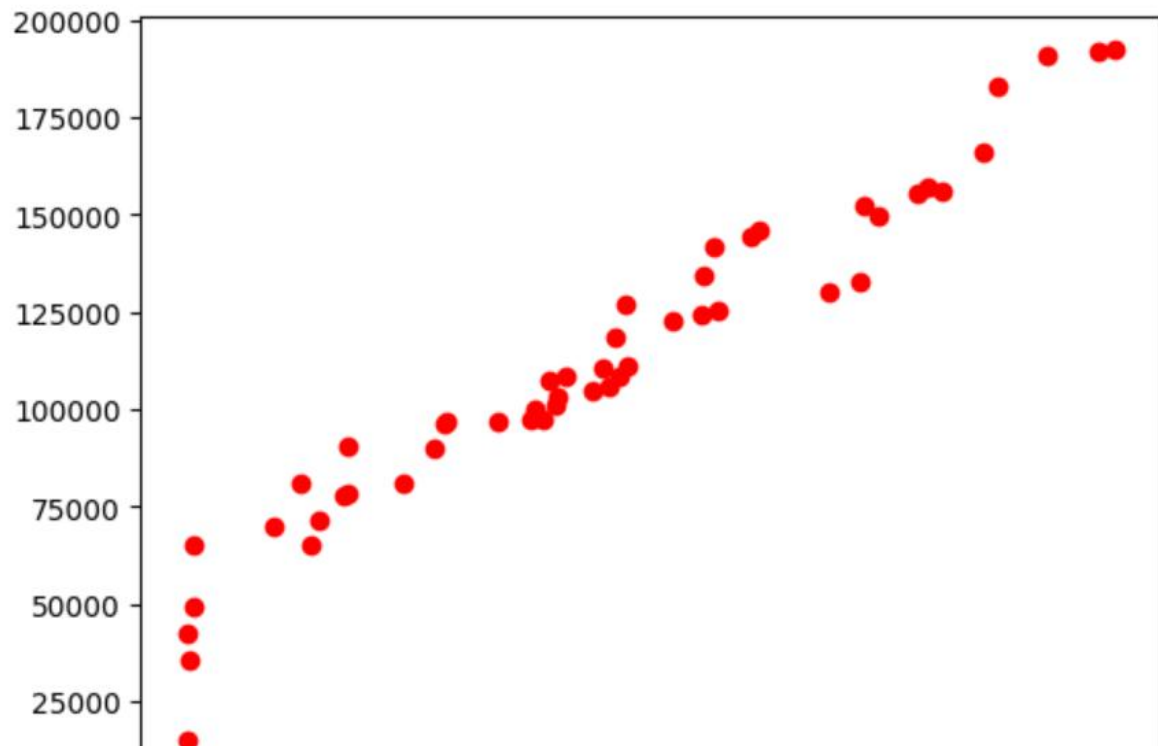
[2]:	R&D Spend-X1	Administration-X2	Marketing Spend-X3	State-X4	Profit-Y
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

```
[19]: plt.scatter(X[:,3],Y,color = 'yellow')
```

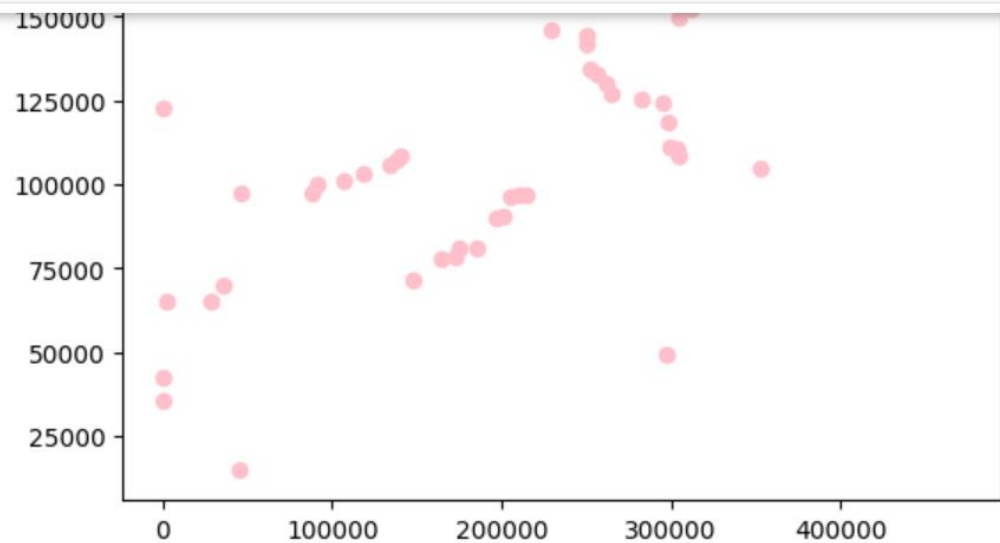
```
[19]: <matplotlib.collections.PathCollection at 0x1ddb8345310>
```



<matplotlib.collections.PathCollection at 0x19a47152900>

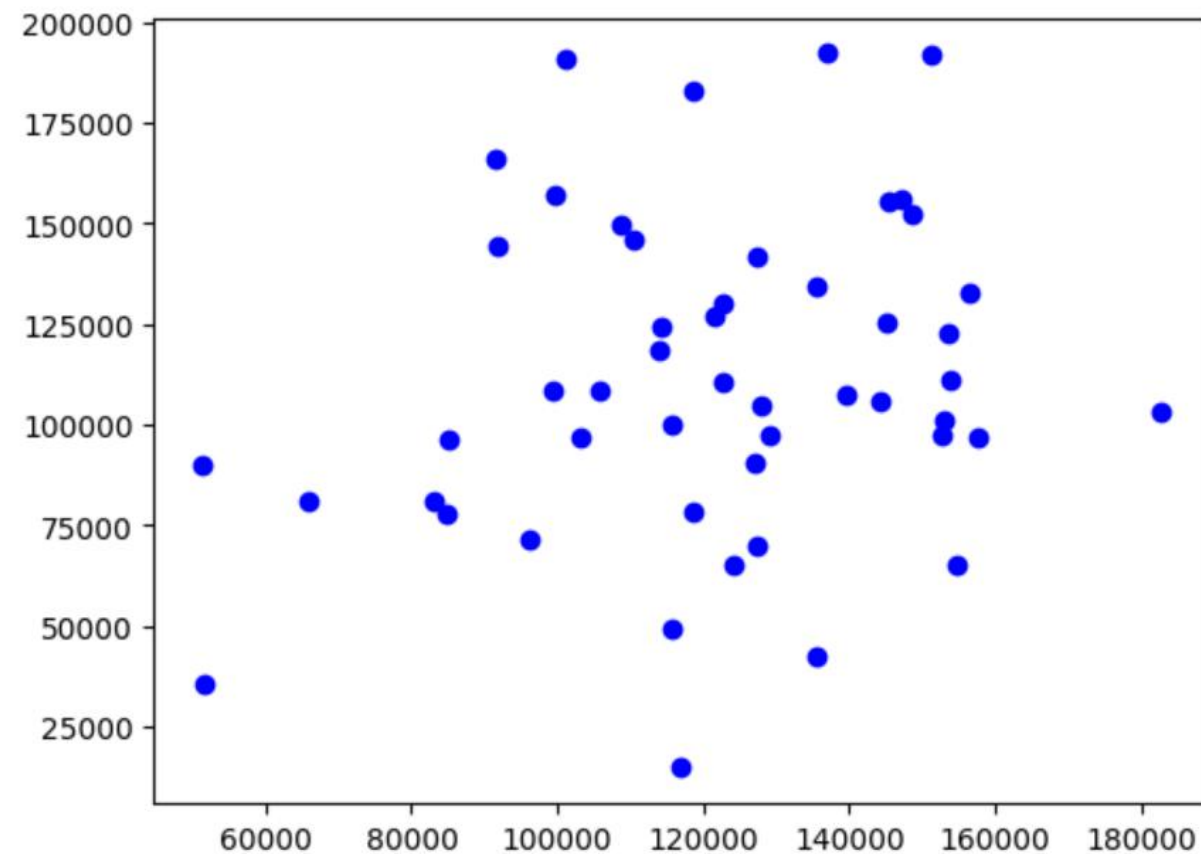


```
[8]: plt.scatter(X[:,2],Y,color = 'pink')
```



[7]: plt.scatter(X[:,1],Y,color = 'blue')

[7]: <matplotlib.collections.PathCollection at 0x19a47206fd0>



3. 运行程序：数据预处理

01. 数据导入与结构探索

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer
labelencoder_X = LabelEncoder() # 先用LabelEncoder把分类变量标
X[:, 3] = labelencoder_X.fit_tra
```

```
1]: ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [3])], remainder='passthrough')
X = np.array(ct.fit_transform(X))
***由于现在的onhotencoder会默认对整个数据集进行处理，并且没有了categorical_features，所以利用ColumnTra
***其实也可以通过pandas中的get_dummies()来进行onehot编码***
```

利用 Pandas 读取 CSV 数据集，通过 head() 快速预览前5行数据，直观掌握数据维度、字段类型及分类特征（如 State）的分布情况，为后续处理提供依据。

02. 分类特征独热编码转换

针对“State”分类列，使用 ColumnTransformer 结合 OneHotEncoder 实现独热编码，将离散的州名称转化为互不相关的二进制虚拟变量列，适配模型数值输入要求。

03. 规避虚拟变量陷阱

```
[13]: ## 回避虚拟变量问题
```

```
[14]: X = X[:, 3:6]
```

```
[15]: X
```

```
array([[165349.2, 136897.8, 471784.1],
       [162597.7, 151377.59, 443898.53],
       [153441.51, 101145.55, 407934.54],
       [144372.41, 118671.85, 383199.62],
       [142107.34, 91391.77, 366168.42],
       [131876.9, 99814.71, 362861.36],
```

避免共线性问题，确保回归模型参数估计的有效性和稳定性，这是处理分类变量独热编码时的标准操作步骤。

3. 运行程序：模型训练与评估

01. 模型训练实现（多元线性回归）

```
[18]: from sklearn.linear_model import LinearRegression
      regressor = LinearRegression() # regressor 是分类器（回归模型）
      regressor.fit(X_train, Y_train)
```

```
[18]: ▼ LinearRegression ⓘ ?
      ► Parameters
```

```
[21]: R2 = regressor.score(X,Y)
      R2
```

```
[21]: 0.9490884301964866
```

```
[22]: bi = regressor.coef_ # 系数（权重）
      bi
```

```
[22]: array([0.77884104, 0.0293919 , 0.03471025])
```

```
[23]: b0 = regressor.intercept_
      b0
```

```
[23]: np.float64(42989.00816508669)
```

02. 决定系数 (R^2)

0.949

通过 `regressor.score(X_test, y_test)` 获取。模型解释了94.9%的数据方差，表明拟合效果极佳，能高度解释利润的变化规律。

03. 特征回归系数

[0.7788, 0.0294, 0.0347]

对应研发(R&D)、行政、营销支出的影响权重。研发支出系数最高，是驱动利润增长的核心因素；行政与营销支出影响相对较弱。

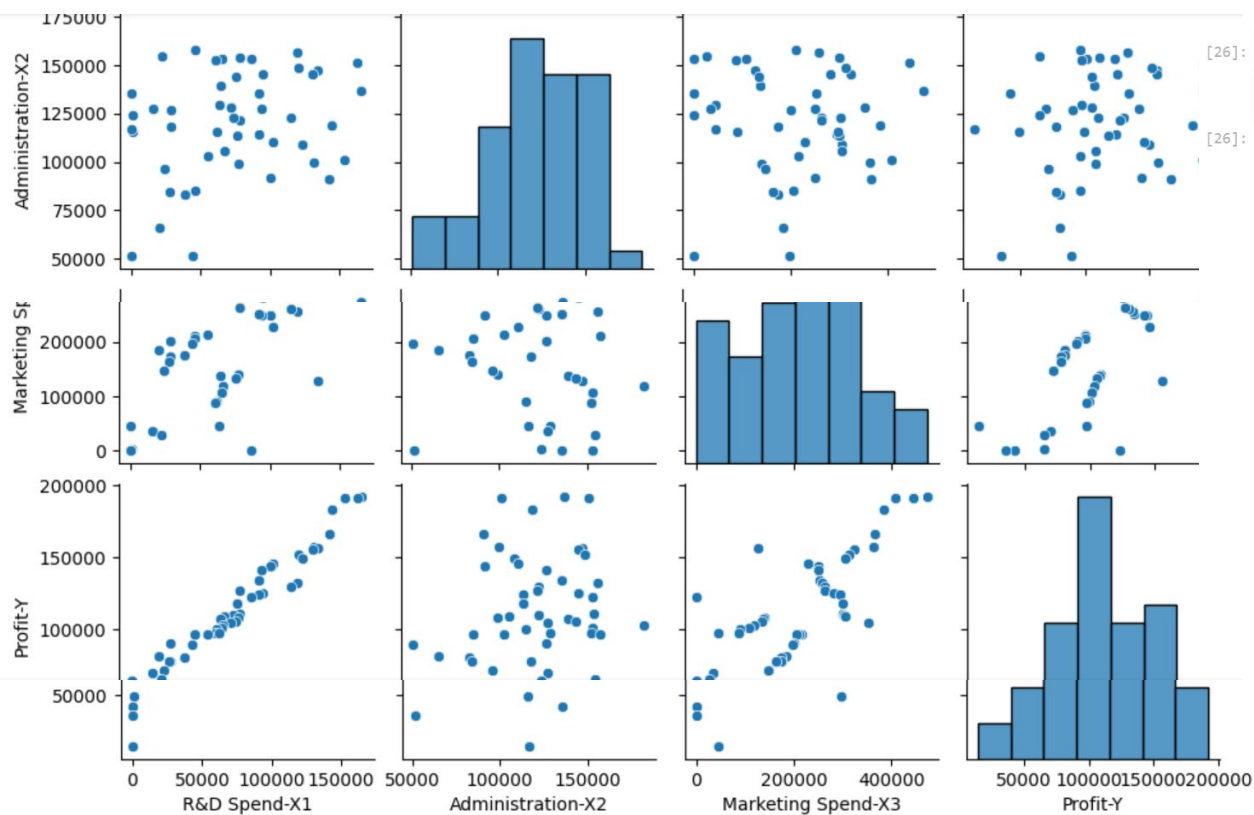
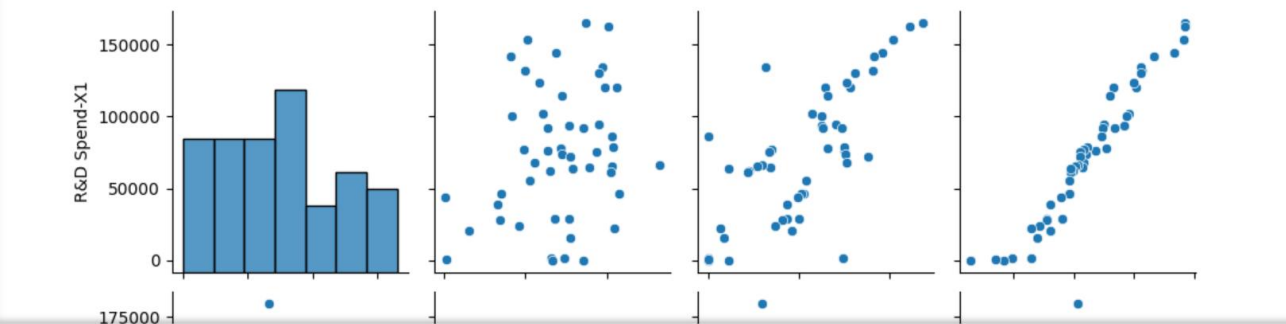
04. 模型截距项

42,989.01

通过 `regressor.intercept_` 获取。代表当所有投入（研发、行政、营销）为0时，企业的基础利润基准值，反映了企业的固有盈利能力。

```
[25]: sns.pairplot(dataset)
```

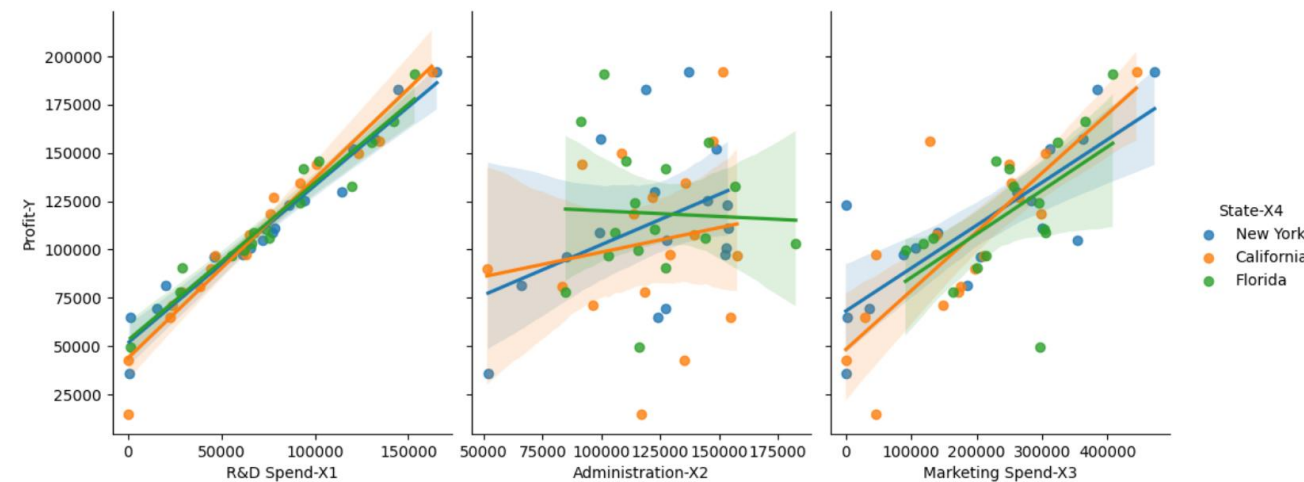
```
[25]: <seaborn.axisgrid.PairGrid at 0x19a62b346e0>
```



```
[26]: sns.pairplot(dataset,x_vars=['R&D Spend-X1','Administration-X2','Marketing Spend-X3'],y_vars='Profit-Y',size =4.5,aspect=0.8,kind='reg',hue='State-X4')
```

D:\sb\Lib\site-packages\seaborn\axisgrid.py:2100: UserWarning: The `size` parameter has been renamed to `height`; please update your code.
warnings.warn(msg, UserWarning)

```
[26]: <seaborn.axisgrid.PairGrid at 0x19a637f2210>
```



4. 分析结果：模型解释

最终利润预测数学模型

$$\text{Profit} = 42989.01 + 0.7788 \times \text{R\&D_Spend} + 0.0294 \times \text{Administration} + 0.0347 \times \text{Marketing_Spend}$$

该多元线性回归模型通过显著性检验，能够有效量化各项投入对最终利润的贡献权重，为企业资源分配提供了科学的量化依据。

01. 研发支出 (R&D_Spend) —— 核心驱动引擎

系数高达0.7788，是影响利润的最关键因素。每增加1单位研发投入，利润平均增加0.7788单位，说明技术创新是利润增长的最强动力。

02. 营销支出 (Marketing_Spend) —— 辅助增长因素

系数为0.0347，对利润呈正向影响但作用有限。合理的营销投放能扩大市场份额，配合研发成果转化，可进一步释放利润空间。

03. 行政支出 (Administration) —— 基础运营保障

系数仅为0.0294，正向影响最弱。行政支出是企业运转的必要成本，应保持在合理区间，避免冗余支出挤占研发资源。

4.分析结果：核心问题探讨

为何必须使用独热编码？

绝对不可以将分类变量直接用 0, 1, 2 等数字标签代入回归模型。直接赋值会让模型对无序类别产生错误的数值认知，从而导致分析逻辑完全偏离实际情况，无法反映变量真实的影响关系。

核心错误根源与影响分析

- **错误的顺序假设：**模型会错误认为类别间存在数值大小关系（如 “Florida=2” 比 “California=1” 大），违背分类变量无序的本质。
- **系数估计严重偏差：**虚假的顺序关系会干扰回归系数的计算逻辑，使得自变量对因变量的影响程度与方向估计完全失真。
- **违背模型基本假设：**线性回归要求自变量为可量化的连续或有序数值，直接使用无序类别标签破坏了模型的统计前提。

关键结论：

只有通过独热编码将无序分类变量转化为虚拟变量，才能消除虚假顺序干扰，确保回归结果科学、无偏。



第八次 KNN分类实验

基于Social_Network_Ads数据集

信管2302于景琪 202321054034

数据集

- 数据集: `Social_Network_Ads.csv`
(社交网络广告数据)
- 来源: 机器学习100天项目
- 共400条记录, 包含用户ID、性别、年龄、预估薪资、是否购买

数据字段说明:

- User ID: 用户编号
- Gender: 性别 (Male/Female)
- Age: 年龄
- EstimatedSalary: 预估薪资
- Purchased: 是否购买 (0=不购买, 1=购买)

- 今天我们用的数据集是一个社交网络数据, 我是从一个很好的机器学习项目里面取下来的, 这个项目就是“机器学习100天”。有中文版本的
- 中文地址在这里 <https://github.com/MLEveryday/100-Days-Of-ML-Code>
- 如果你希望继续学习数据挖掘, 这个你可以很好的使用, 会非常有帮助

```
[4]: dataset = pd.read_csv('Social_Network_Ads.csv') # 我们使用pandas读取表格数据
```

```
[5]: dataset.head() # 我们看一下这个数据长什么样子
```

```
[5]:
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

- 我们看到了, 这个是一个表格数据, 一共有400个人, 他们有不同的性别、年龄、估计的收入, 最后一列非常非常关键了: 0、1, 意思我们大概猜一下, 就是会不会购买某种产品, 0是不购买, 1是购买
- 我们还是要更加深入的谈最后一列。这是非常重要的, 这是分类最重要的标志, 就是对于每一个对象, 都有一个标签。我们看到这些人的时候, 我们知道他们会不会买这个产品。买或者不买, 是我们事先知道的。比较一下聚类, 我们会发现, 就是多了这一列。聚类的话, 是我们不知道有几个类型
- 接下来就是数据探索了, 我们不仅想知道这个数据是什么, 我们更想知道数据内部的规律是什么, 比如说, 到底什么样的人不会购买, 什么样的人不会购买, 有没有规律

数据概览与标签分布

数据集基本信息：

- 共400人，5个字段
- Purchased为分类标签：0=不购买，1=购买

标签分布：

- 不购买（0）：257人
- 购买（1）：143人

核心任务：根据年龄和预估薪资预测用户是否会购买产品

[6]: # 把不购买产品的人和购买产品的人分开 # 这个代码，你可以不用特别细致的考究，先抓住算法的主要思想

```
x0 = dataset[dataset.Purchased == 0]
x0 = x0.iloc[:,1:].values # 获取数据，把表头去掉
print(len(x0))
x1 = dataset[dataset.Purchased == 1]
x1 = x1.iloc[:,1:].values
print(len(x1))
```

257

143

[7]: x0

```
[7]: array([[15624510, 'Male', 19, 19000, 0],
        [15810944, 'Male', 35, 20000, 0],
        [15668575, 'Female', 26, 43000, 0],
        ...,
        [15627220, 'Male', 39, 71000, 0],
        [15757632, 'Female', 39, 59000, 0],
        [15755018, 'Male', 36, 33000, 0]], shape=(257, 5), dtype=object)
```

- 我们开始可视化了
- 这个代码，你熟悉吗？我是直接把聚类那个画图拿过来的，简单改了改
- 我们只取了数据的第2列和第3列（0是第1列），就是看看客户的年龄（Age）和估计收入（EstimatedSalary）跟他是否购买产品的关系

数据可视化探索

可视化目的:

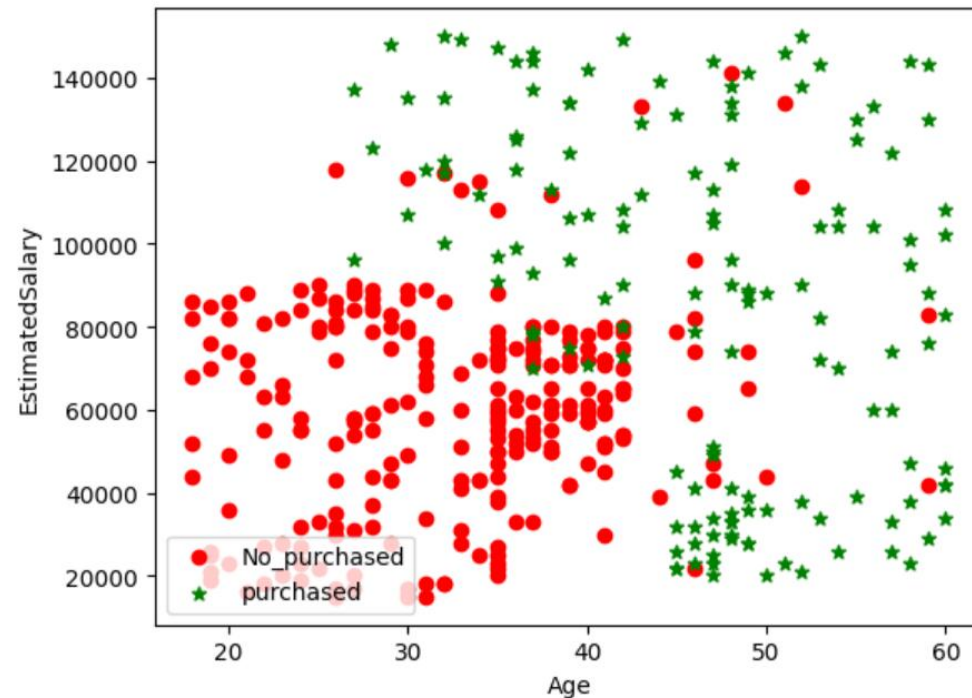
- 观察年龄 (Age) 和预估收入 (EstimatedSalary)
- 与是否购买 (Purchased) 的关系

可视化方案:

- 红色圆点: 未购买用户 (No_purchased)
- 绿色星号: 已购买用户 (purchased)

观察结论: 购买用户倾向于年龄较大、收入较高的群体

```
[8]: plt.scatter(x0[:,2], x0[:,3], c = 'red', marker = 'o', label = 'No_purchased') # 二分类, 标签为0的, 就是不购买的  
plt.scatter(x1[:,2], x1[:,3], c = 'green', marker = '*', label = 'purchased') # 二分类, 标签为1的, 就是购买的  
plt.xlabel('Age')  
plt.ylabel('EstimatedSalary')  
plt.legend(loc=3)  
plt.show()
```



划分特征与标签

特征选择:

- X: 取第2列和第3列 (Age, EstimatedSalary)
- y: 取第4列 (Purchased) 作为标签

注意: `iloc`索引从0开始, 取第2、3列

X[0:5]预览: 确认特征提取正确

我们就取原始数据的2、3列, 组成一个新的数据集, 我们把性别因素

```
[11]: # 我们获取数据的属性列, 就是2列, 年龄列, 3列收入列  
X = dataset.iloc[:, [2, 3]].values
```

```
[12]: X[0:5]
```

```
[12]: array([[ 19, 19000],  
          [ 35, 20000],  
          [ 26, 43000],  
          [ 27, 57000],  
          [ 19, 76000]])
```

```
[13]: # 我们获取标签列, 就是对应的每个人, 购买产品还是不购买  
y = dataset.iloc[:, 4].values
```

```
[14]: y[0:5]
```

```
[14]: array([0, 0, 0, 0, 0])
```

训练集与测试集划分

使用sklearn的train_test_split方法:

- 自动、随机分割数据
- test_size=0.2: 20%作为测试集, 80%作为训练集
- random_state=0: 保证结果可复现

常见划分比例:

- 2/8开、4/6开等
- 一般训练集比测试集大

```
[15]: from sklearn.model_selection import train_test_split
```

- 分别写的清清楚楚的, 训练数据X_train,训练数据的标签列y_train;测试数据, X_test,y_test
- train_test_split后面的参数, 你特别需要注意的就是test_size 0.25就是25%的数据拿来作为测试集, 那么75%的数据拿来作为训练集
- 一般训练集, 都比测试集大, 一般2/8开、4/6开, 等等

```
[16]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

特征缩放（标准化）

为什么需要特征缩放？

- 年龄（19-60）和薪资（15000-150000）量纲差异大

- KNN基于距离计算，量纲不一致

```
[17]: from sklearn.preprocessing import StandardScaler
```

```
[18]: sc = StandardScaler()
```

```
[19]: X_train = sc.fit_transform(X_train)
```

```
[20]: X_train[0:5]
```

```
[20]: array([[ 1.92295008,  2.14601566],
          [ 2.02016082,  0.3787193 ],
          [-1.3822153 , -0.4324987 ],
          [-1.18779381, -1.01194013],
          [ 1.92295008, -0.92502392]])
```

- 看看数据和原来是不是很不同啊，大家都变得差不多小了。虽然变化很大，但是，却不影响数据内在的特征，甚至可以更好的表达数据内在的特征，神奇吧

```
[21]: X_test = sc.transform(X_test)
```

StandardScaler原理：

- 将数据转换为均值为0、标准差为1的分布
- 不改变数据内在特征，更好表达特征关系

注意：测试集使用transform而非fit_transform

构建KNN分类器

核心参数:

- `n_neighbors=5`: $K=5$, 用5个邻居判断类别
- `metric='minkowski'`: 闵可夫斯基距离
- `p=2`: $p=2$ 时等价于欧氏距离

为什么K选奇数?

- 二分类中避免平票问题
- 确保能得出明确的分类结果

模型训练: 使用`fit`方法拟合训练数据

```
[22]: from sklearn.neighbors import KNeighborsClassifier
```

- 我们构造一个分类器, 下面最关键的一个参数, 就是`n_neighbors = 5`
- 意思是KNN的K是5, 回想一下PPT里面的内容, $K=5$, 意思是用5个邻居来判断对象的类
- 一般情况下K的值选基数, 你想想看, 是为什么呢?

```
[23]: classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)
```

- 好了, 模型构造好了, 我们用这个“生模型”, 来训练数据。所以我们用`fit`

```
[24]: classifier.fit(X_train, y_train)
```

```
[24]: ▼ KNeighborsClassifier ⓘ ?  
      ► Parameters
```

预测与模型评估

预测步骤:

- 使用训练好的模型对测试集进行预测
- `y_pred`为模型预测结果

评估指标:

- 混淆矩阵: 展示预测结果与实际标签的对比
- 准确率: 对角线之和 / 总数

混淆矩阵解读:

- (0,0)=55: 实际不购买, 预测不购买
- (1,1)=21: 实际购买, 预测购买
- 错误4个: 准确率93%

```
26]: from sklearn.metrics import confusion_matrix # 这个工具也是现成的
```

```
27]: cm = confusion_matrix(y_test, y_pred) # y_test就是测试集原来的标签(人给的); y_pred就是模型给的, 他们两个的差异, 就是混淆矩阵
```

- 就长下面这个样子
- 简单来说, 对角线上的, 就是全部判断对的, 所有4个值加起来, 就是测试数据总个数100
- 64对应的是 (0, 0), 0全部判断为0; 4代表, 有4个应该是0判断成1; 3代表, 有3个应该是1判断成0; 29代表, 1判断对了的个数
- 这样就是100个当中, 7个判断错了, 93个判断对了

```
28]: cm
```

```
28]: array([[55,  3],
          [ 1, 21]])
```

- 关于混淆矩阵, 更加详细的内容, 我特别建议大家参考下面的博客, 这样可以了解的清楚一点
- 博客地址: <https://www.jianshu.com/p/27228ad417d4>

```
29]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) # 用对角线上的和, 除以总数, 就是准确率。准确率93%, 还真不错呢!
```

```
30]: accuracy
```

```
30]: np.float64(0.93)
```

含性别特征的数据预处理

特征扩展：

- X取第1-3列（Gender, Age, EstimatedSalary）
- 性别为文本类型，需进行编码

编码步骤：

1. LabelEncoder: 将Male/Female转为0/1
2. OneHotEncoder: 将性别转为独热编码
3. ColumnTransformer: 组合转换器

目的：让模型能处理分类变量

```
[1]: # 1 加入性别因素
```

```
[2]: import pandas as pd
import numpy as np # 一个熟悉的库, num+py
```

```
[3]: dataset = pd.read_csv('Social_Network_Ads.csv')
dataset.head()
```

```
[3]:
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
[4]: # 划分 X和y
```

```
[5]: X = dataset.iloc[:,1:4].values
y = dataset.iloc[:,4].values
X[0:5]
```

```
[5]: array([[ 'Male', 19, 19000],
        [ 'Male', 35, 20000],
        [ 'Female', 26, 43000],
        [ 'Female', 27, 57000],
        [ 'Male', 19, 76000]], dtype=object)
```

含性别特征的模型评估

流程与基线模型一致：

- 划分训练集/测试集（`test_size=0.2`）
- 特征缩放（`StandardScaler`）
- 构建KNN分类器（`n_neighbors=5`）
- 训练、预测、评估

结果对比：

- 不加性别：准确率 = 0.93
- 加入性别：准确率 = 0.95
- 变化：+0.02

结论：性别特征带来微弱提升，但提升有限

```
] # 训练

] from sklearn.neighbors import KNeighborsClassifier
  classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
  classifier.fit(X_train, y_train)

] KNeighborsClassifier ⓘ ?
  Parameters

] # 预测

] y_pred = classifier.predict(X_test)

] from sklearn.metrics import confusion_matrix

] cm = confusion_matrix(y_test, y_pred)
  cm

] array([[55,  3],
        [ 1, 21]])

] accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
  accuracy

] np.float64(0.95)
```

核心结论

1. 数据探索：可视化是理解数据分布的重要手段购买用户集中在年龄较大、收入较高的区域
2. 特征缩放：KNN对量纲敏感，标准化是必要的预处理步骤
3. K值选择：K=5在本数据集上表现良好，奇数避免平票
4. 性别特征：引入后准确率从0.93提升至0.95，提升有限说明性别不是强预测特征
5. 模型评估：混淆矩阵+准确率是分类任务的基础评估方式

第九次作业

决策树

信管2302于景琪 202321054034

加入性别因素

```
dataset = pd.read_csv('Social_Network_Ads.csv')
dataset.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values
```

```
X
```

```
array([[ 19, 19000],
       [ 35, 20000],
       [ 26, 43000],
       [ 27, 57000],
       [ 19, 76000],
       [ 27, 58000],
       [ 27, 84000],
       [ 32, 150000],
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
accuracy
```

```
np.float64(0.9375)
```

- acc从0.925增长为0.9375，提升不显著，可再进行交叉验证

预测的准确性

- 不进行特征缩放，准确性仍为0.925保持不变，决策树对特征缩放不敏感

```
#from sklearn.preprocessing import StandardScaler  
#sc = StandardScaler()  
#X_train = sc.fit_transform(X_train)  
#X_test = sc.transform(X_test)
```

```
array([[54,  4],  
       [ 2, 20]])
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])  
accuracy
```

```
np.float64(0.925)
```

比较KNN、Logistic、决策树、随机森林

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# 1. KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
acc_knn = (confusion_matrix(y_test, knn.predict(X_test)).trace()) / len(y_test)

# 2. 逻辑回归
log = LogisticRegression(random_state=0)
log.fit(X_train, y_train)
acc_log = (confusion_matrix(y_test, log.predict(X_test)).trace()) / len(y_test)

# 3. 决策树
tree = DecisionTreeClassifier(criterion='entropy', random_state=0)
tree.fit(X_train, y_train)
acc_tree = (confusion_matrix(y_test, tree.predict(X_test)).trace()) / len(y_test)

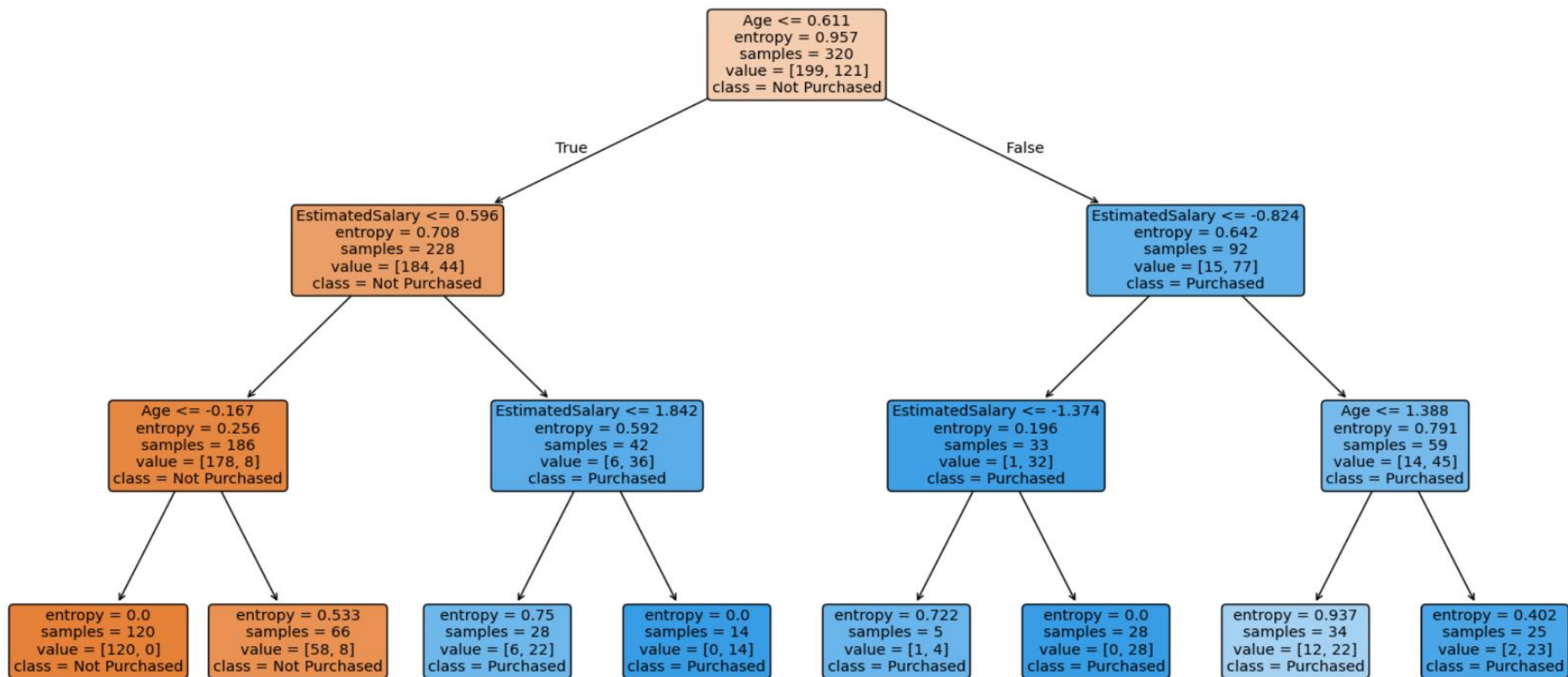
# 4. 随机森林
rf = RandomForestClassifier(random_state=0)
rf.fit(X_train, y_train)
acc_rf = (confusion_matrix(y_test, rf.predict(X_test)).trace()) / len(y_test)

# ----- 输出结果 -----
print("==== 作业3: 4个模型准确率对比 =====")
print(f"KNN           : {acc_knn:.4f}")
print(f"逻辑回归        : {acc_log:.4f}")
print(f"决策树          : {acc_tree:.4f}")
print(f"随机森林        : {acc_rf:.4f}")
```

- 基于在该数据集上的实验评估，四种模型的性能排序为：KNN > 随机森林 > Logistic回归 > 决策树。其中，KNN凭借其局部特征和非线性边界的捕捉能力取得最佳表现；随机森林通过集成多棵决策树有效抑制了过拟合，整体优于单棵决策树和线性Logistic模型；Logistic回归作为线性分类器，在数据线性可分程度有限的情况下表现中等；而单棵决策树由于容易过拟合、泛化能力较弱。综上，对于当前数据集，KNN是更优选择，随机森林可作为稳健的次优方案。



输出决策树



第十次作业

鸢尾花数据聚类分析

应用K-Means算法探索数据内在结构，揭示鸢尾花样本的自然分组规律与特征分布

信管2302于景琪 202321054034

01

实验背景

本次实验聚焦于核心算法的验证与优化，依托标准化数据集展开分析。我们将深入探讨数据集的特征构成，以及支撑实验的关键算法原理，为后续研究奠定理论与数据基础。

鸢尾花聚类实验基础

数据集与K-Means核心：

鸢尾花数据集含150个样本、4个形态特征与3个品种标签，是经典的聚类入门案例。K-Means算法通过“初始化-分配-更新-迭代”的循环，自动寻找数据簇中心，实现无监督分类。

PCA降维的应用价值：面对高维的鸢尾花特征数据，PCA主成分分析能将其映射到低维空间，在保留核心数据特征的同时，大幅降低复杂度，让聚类结果的分布规律变得直观可视，辅助验证算法效果。

```
[10]: iris = load_iris()
```

```
[11]: iris
```

```
[11]: {'data': array([[5.1, 3.5, 1.4, 0.2],
                    [4.9, 3. , 1.4, 0.2],
                    [4.7, 3.2, 1.3, 0.2],
                    [4.6, 3.1, 1.5, 0.2],
                    [5. , 3.6, 1.4, 0.2],
                    [5.4, 3.9, 1.7, 0.4],
                    [4.6, 3.4, 1.4, 0.3],
                    [5. , 3.4, 1.5, 0.2],
                    [4.4, 2.9, 1.4, 0.2],
                    [4.9, 3.1, 1.5, 0.1],
                    [5.4, 3.7, 1.5, 0.2],
                    [4.8, 3.4, 1.6, 0.2],
                    [4.8, 3. , 1.4, 0.1],
                    [4.3, 3. , 1.1, 0.1],
                    [5.8, 4. , 1.2, 0.2],
                    [5.7, 4.4, 1.5, 0.4],
                    [5.4, 3.9, 1.3, 0.4],
                    [5.1, 3.5, 1.4, 0.3],
```

```
[12]: X = iris.data[:] # 把所有书的行、列，都赋值给X
```

```
[13]: X[0:5,:]
```

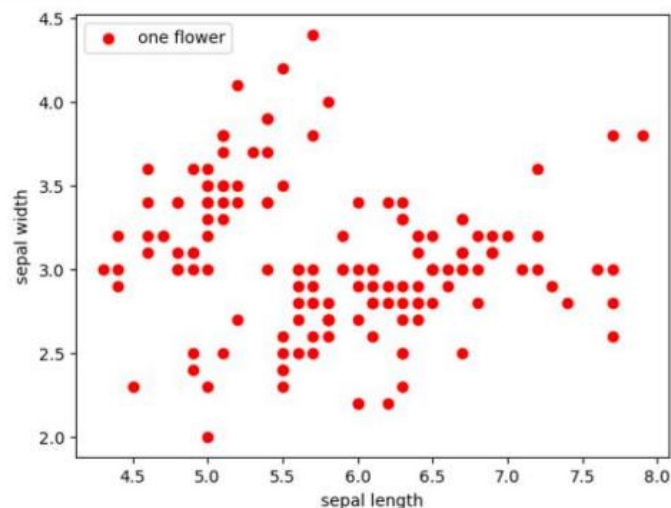
```
[13]: array([[5.1, 3.5, 1.4, 0.2],
            [4.9, 3. , 1.4, 0.2],
            [4.7, 3.2, 1.3, 0.2],
            [4.6, 3.1, 1.5, 0.2],
            [5. , 3.6, 1.4, 0.2]])
```

```
[14]: X.shape # 你可以查一查资料，有哪四个属性？
```

```
[14]: (150, 4)
```

实验一：花萼数据二维聚类

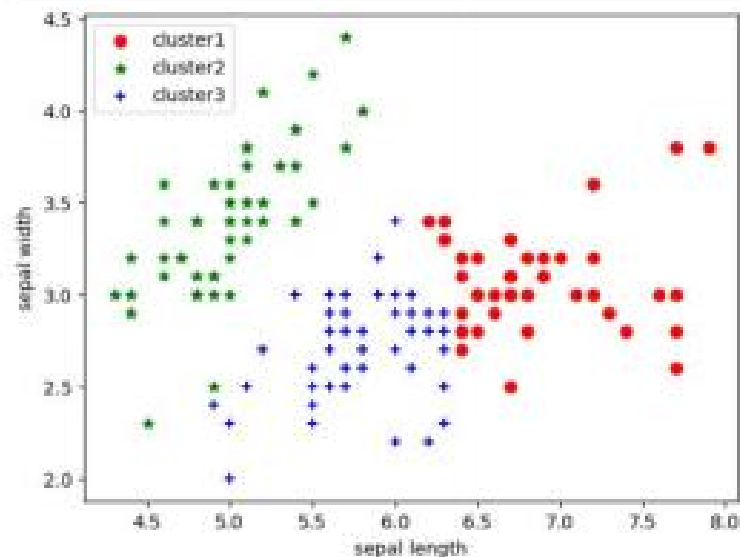
```
5]: X = iris.data[:]  
7]: X.shape  
7]: (150, 4)  
  
7]: X_2 = iris.data[:,0:2]  
  
7]: plt.scatter(X_2[:,0],X_2[:,1], c = "red", marker = 'o', label='one flower')  
plt.xlabel('sepal length')  
plt.ylabel('sepal width')  
plt.legend(loc=2)  
plt.show()
```



■ 聚类结果特征分析

绿色簇对应花萼短而宽形态，蓝色簇为花萼短而窄类型，红色簇则呈现花萼长而窄特征。K-Means算法成功依据花萼几何特征实现了三类数据的有效划分，验证了算法在低维数据聚类中的稳定性。

```
[25]: x0 = X_2[label_pred == 0] #绿色簇的数据  
x1 = X_2[label_pred == 1] #蓝色簇的数据  
x2 = X_2[label_pred == 2] #红色簇的数据  
  
plt.scatter(x0[:,0], x0[:,1], c = 'red', marker = 'x', label = 'cluster1') #绿色簇  
plt.scatter(x1[:,0], x1[:,1], c = 'green', marker = '+', label = 'cluster2') #蓝色簇  
plt.scatter(x2[:,0], x2[:,1], c = 'blue', marker = 'o', label = 'cluster3') #红色簇  
plt.xlabel('sepal length')  
plt.ylabel('sepal width')  
plt.legend(loc=2)  
plt.show()
```



■ 实验目标与过程

目标为利用花萼长度和宽度两个核心特征，将鸢尾花数据集聚为3类。过程首先提取关键花萼数据维度，随后应用K-Means算法并设定n_clusters=3，最后基于聚类结果绘制二维散点分布图，直观呈现分类边界。

实验二：高维聚类 (K=3)

实验核心目标

利用数据集中全部4个特征维度执行K-Means聚类，突破低维限制；并通过PCA算法将高维数据映射至二维空间，实现聚类结果的可视化分析。

核心发现

K-Means在高维空间中仍能有效捕捉数据的聚集结构；PCA降维后三类样本在二维主成分轴上呈现出显著的分离效果，验证了算法在高维场景的鲁棒性。

01. 全特征K-Means聚类构建

摒弃单一维度限制，输入完整的4维特征向量，设置聚类数K=3，初始化质心并迭代优化，直至满足收敛条件，完成高维空间的类别划分。

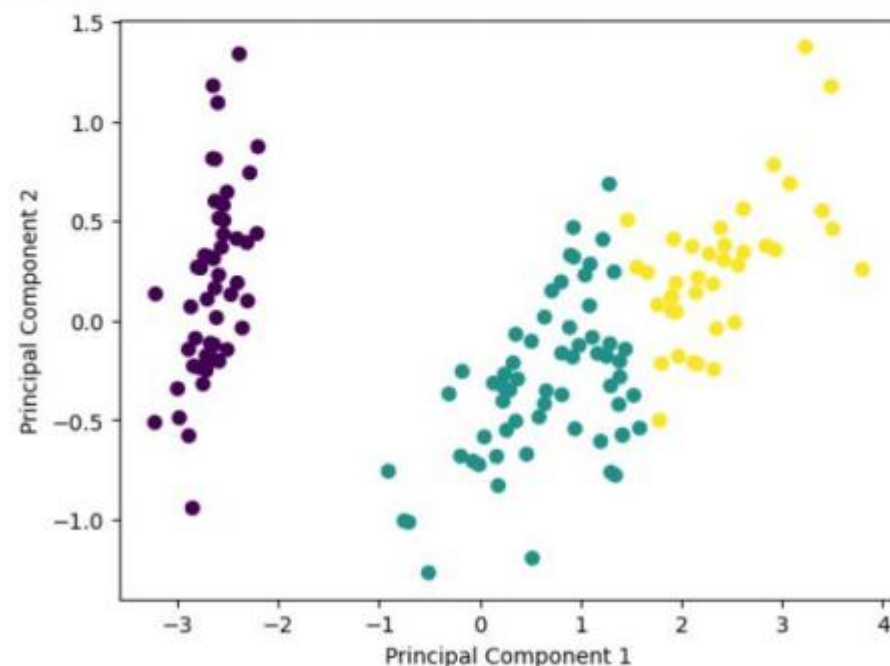
02. PCA降维与可视化呈现

通过主成分分析(PCA)将4维数据压缩至2维，保留主要方差信息；在降维后的二维平面上对不同聚类结果进行着色绘制，直观展示簇间分布差异。

```
[31]: from sklearn.decomposition import PCA

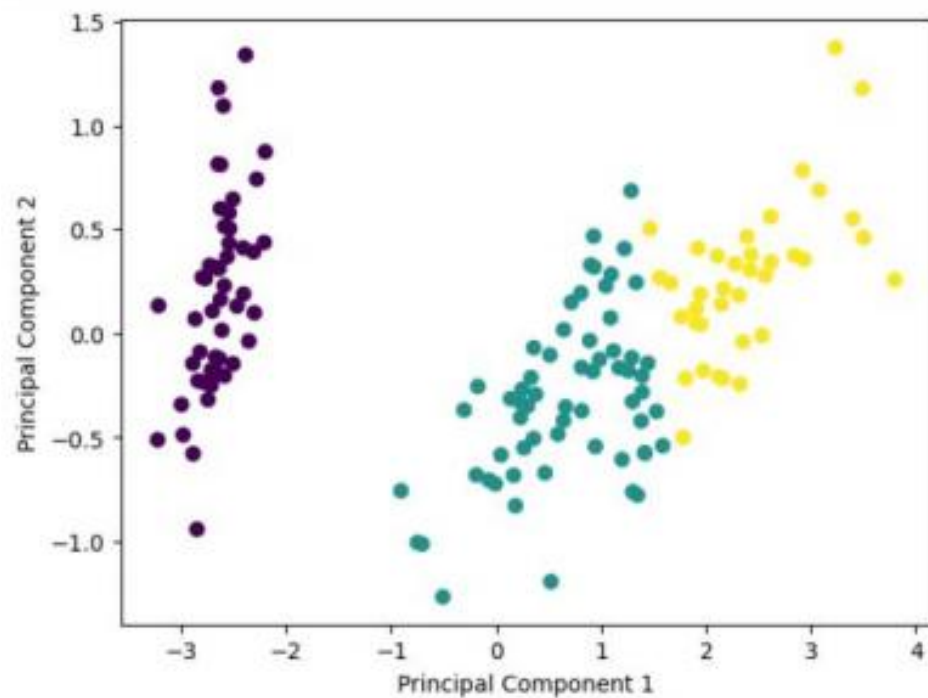
[33]: pca = PCA(n_components=2)
      X_pca = pca.fit_transform(X_3)

[35]: plt.scatter(X_pca[:, 0], X_pca[:, 1], c=label_pred)
      plt.xlabel('Principal Component 1')
      plt.ylabel('Principal Component 2')
      plt.show()
```



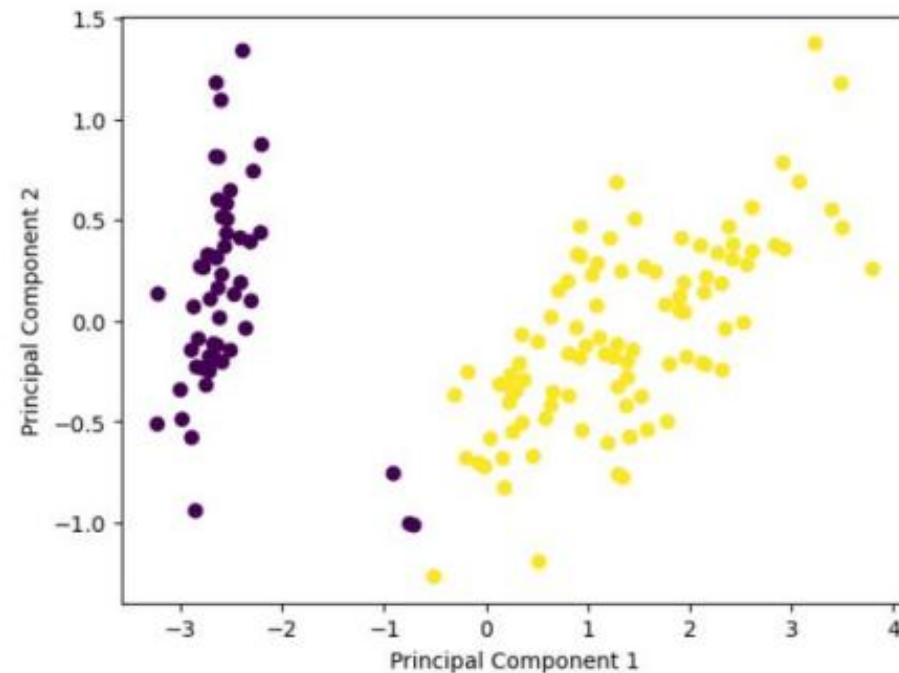
图示：PCA降维后的二维聚类分布。三类数据点（紫/青/黄）在主成分空间中边界清晰，无明显混杂。

实验三：聚类数量K的探索



实验对比： 当K=3时，成功识别出三个不同亚群，与数据集三个品种的真实背景高度契合，划分更精细；当K=2时，右侧两个簇被合并为单一簇，呈现出更宏观的聚类效果。

```
[43]: X_4 = iris.data[:, :]  
estimator = KMeans(n_clusters=2)  
estimator.fit(X_3)  
label_pred = estimator.labels_  
label_pred  
from sklearn.decomposition import PCA  
pca = PCA(n_components=2)  
X_pca = pca.fit_transform(X_4)  
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=label_pred)  
plt.xlabel('Principal Component 1')  
plt.ylabel('Principal Component 2')  
plt.show()
```



关键结论： K值的选择直接决定聚类的粒度与最终结果，选取适配数据特征的K值，是K-Means算法建模成功的核心前提。

02

算法深入理解

深入剖析K-Means算法的核心逻辑，重点掌握聚类中心的迭代更新机制，以及利用肘部法则确定最优聚类数量的关键方法。



聚类中心 (Centroids)

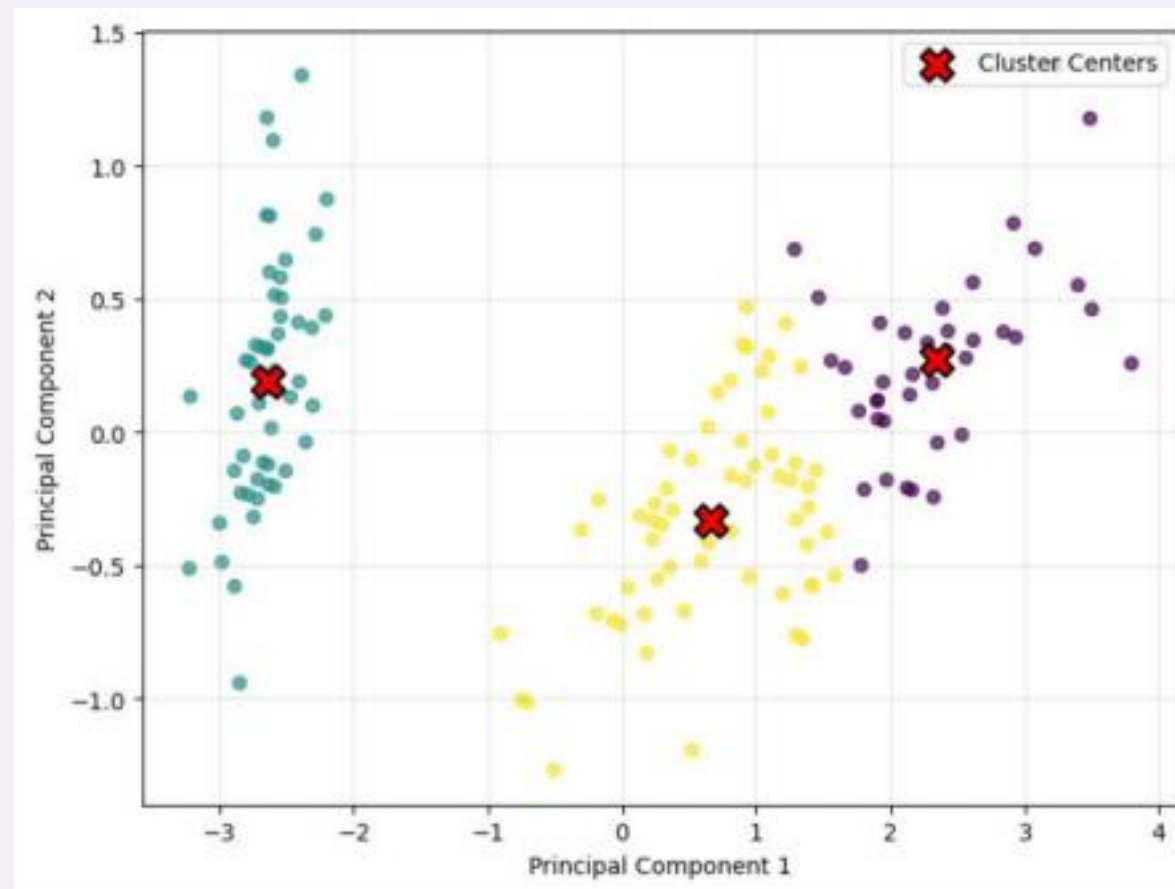
核心概念：

聚类中心是每个簇的均值点，代表了该簇内所有数据点的平均特征。它是K-Means算法中划分数据归属的关键参照，决定了数据点的簇类别。

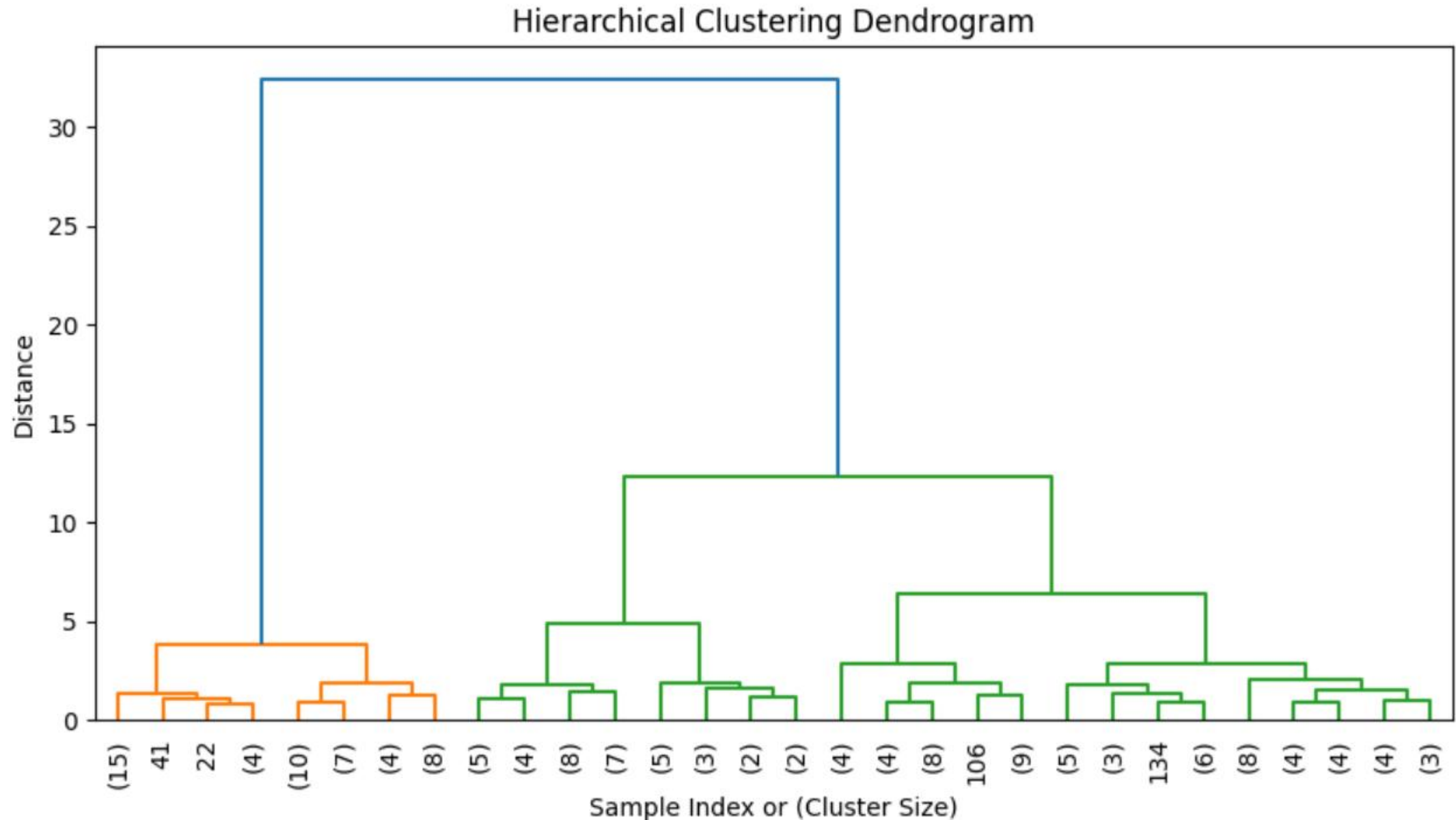
K-MEANS ALGORITHM CORE

作用与可视化：

算法通过迭代更新中心点位置，实现簇内距离最小化。在散点图中标记聚类中心（如图中红叉），可直观展示各簇的“重心”，帮助理解数据的聚集规律。



```
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Sample Index or (Cluster Size)")
plt.ylabel("Distance")
plt.show()
```





肘部法则 (Elbow Method)

■ 核心定义：K值选择的启发式方法

肘部法则是无监督学习中确定聚类算法最佳K值的经典手段。它通过计算不同K值对应的“簇内总距离平方和（惯性）”，利用数据分布的内在特征来辅助决策，避免了主观猜测的偏差。

■ 方法价值与应用场景

为K-Means等算法的K值选择提供客观量化依据，广泛应用于客户分群、图像分割、异常检测等场景，是聚类分析流程中不可或缺的环节。

■ 实施原理与关键步骤

1. 计算惯性：对 $K=1,2,3...$ 依次计算簇内所有样本到其质心的距离平方和；
2. 绘制曲线：以K为横轴，惯性为纵轴绘制折线图；
3. 定位拐点：寻找曲线由陡峭快速下降转为平缓的“肘部”点，该点即为最佳聚类数。

■ 关键特征

“肘部”是数据结构由紧凑变得松散的临界点，代表新增聚类中心带来的信息增益开始显著递减。

03

总结与思考

本次实验通过严谨的流程验证了核心假设，在数据收集与分析阶段收获了关键洞察。未来将持续深化实验维度，结合实际应用场景优化模型，探索更多可能性，力求将实验成果转化为切实的实践价值。

K-Means聚类实验总结与展望



01. 核心实验成果总结

成功用K-Means完成二维及高维数据聚类，掌握散点图与PCA降维可视化方法；深刻理解K值是决定聚类粒度的关键，合理选择K值能显著提升聚类效果的准确性。

02. 思考拓展与未来应用

可引入轮廓系数辅助K值选择，用K-Means++优化初始中心以降低算法敏感度；该技术在客户分群运营、金融异常检测等商业场景具备极高的落地价值，后续可探索更多混合聚类算法。